



使用 Graph RAG、ADK 和 Memory Bank 建構多模態 AI 代理

來源：<https://codelabs.developers.google.com/codelabs/survivor-network/instructions?hl=zh-tw>

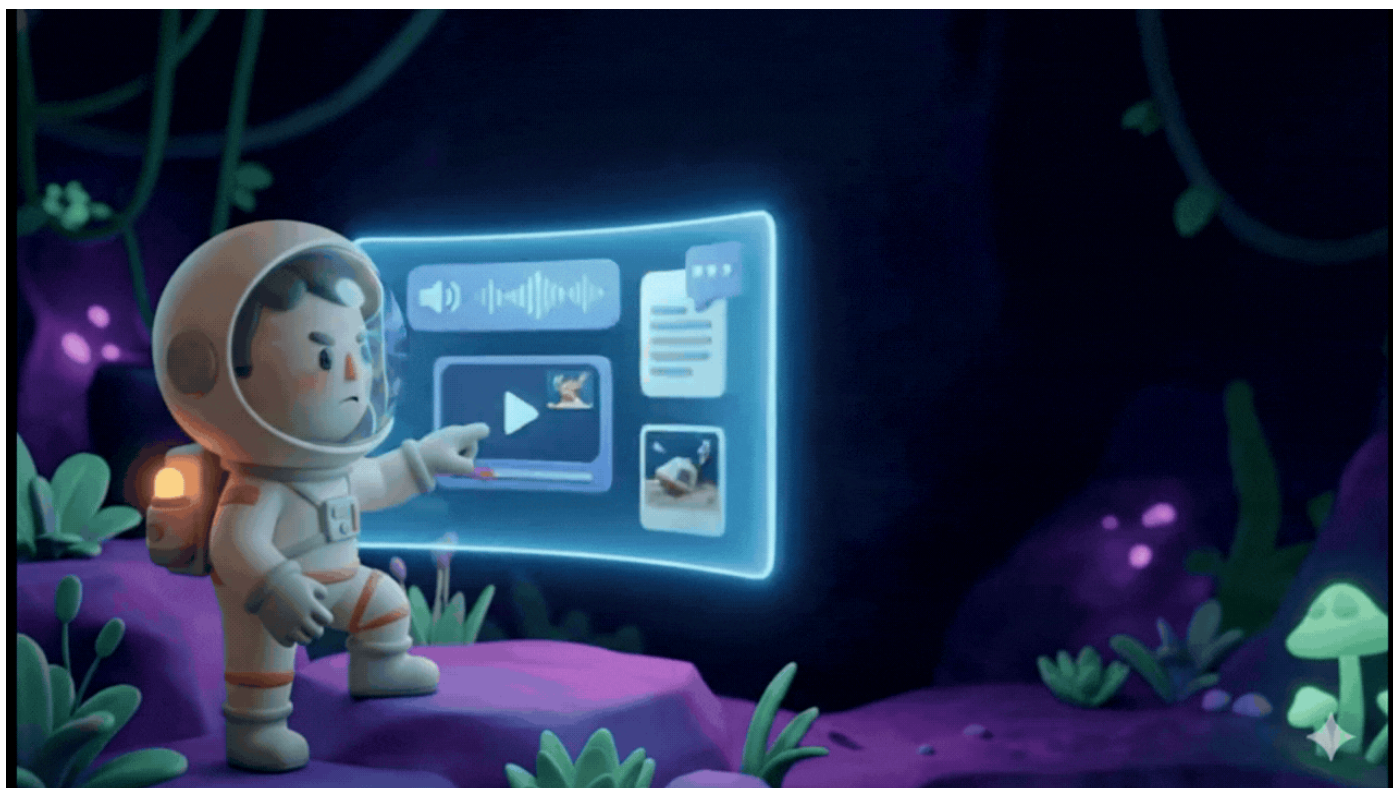
步驟數：15

瞭解如何使用 Spanner Graph、RAG 搜尋、多模態處理和 Memory Bank，建構生產環境等級的 AI 代理程式系統，以利智慧型災害應變協調。

目錄

1. [1. 簡介](#)
2. [2. !\[\]\(467d80e979964f7f8c752fb22248b5b7_img.jpg\) 準備環境 \(如果您在研討會中，請略過此步驟\)](#)
3. [3. !\[\]\(b71552d33dbf62adf5e5199a70ee02bf_img.jpg\) 環境設定](#)
4. [4. !\[\]\(03134b765d1473836ff001925b1b0550_img.jpg\) 在 Spanner Studio 中以圖表呈現圖形資料](#)
5. [5. !\[\]\(aed6947356668967079310026052edc0_img.jpg\) Spanner 中的 AI 輔助嵌入](#)
6. [6. !\[\]\(e61aeb0d9066d5d9e54d9b655f50da3d_img.jpg\) 使用混合搜尋建構 Graph RAG 代理程式](#)
7. [7. !\[\]\(f7af41ce0777e13bda91fa715111c02a_img.jpg\) 使用 ADK Web 測試代理](#)
8. [8. !\[\]\(476ddb2354d4ad1cb23a2236b1e49873_img.jpg\) 執行完整應用程式](#)
9. [9. !\[\]\(1d505a46c82c5cefa23b88c2eee900ce_img.jpg\) \[Optional\] Multimodal Pipeline \(Read Only\) — Tooling Layer](#)
10. [10. !\[\]\(3a98690f11ee4baf67262bd776464219_img.jpg\) \[Optional\] Multimodal Pipeline \(Read Only\) — Agent Layer](#)
11. [11. !\[\]\(35522fe6386206890679adb7b63391b6_img.jpg\) 多模態資料管道 - 自動化調度管理](#)
12. [12. !\[\]\(d28d4a3445dac344f03b5cebc14c5170_img.jpg\) \[Optional\] Memory Bank with Agent Engine](#)
13. [13. !\[\]\(3e37ae08976ee7fa41b108254fcb66a7_img.jpg\) \[選用\] 將代理程式與 Agent Engine 連結](#)
14. [14. !\[\]\(7b30e10e474a15019e378034a5556dd2_img.jpg\) \[選用\] 部署至 Cloud Run](#)
15. [15. 結論](#)

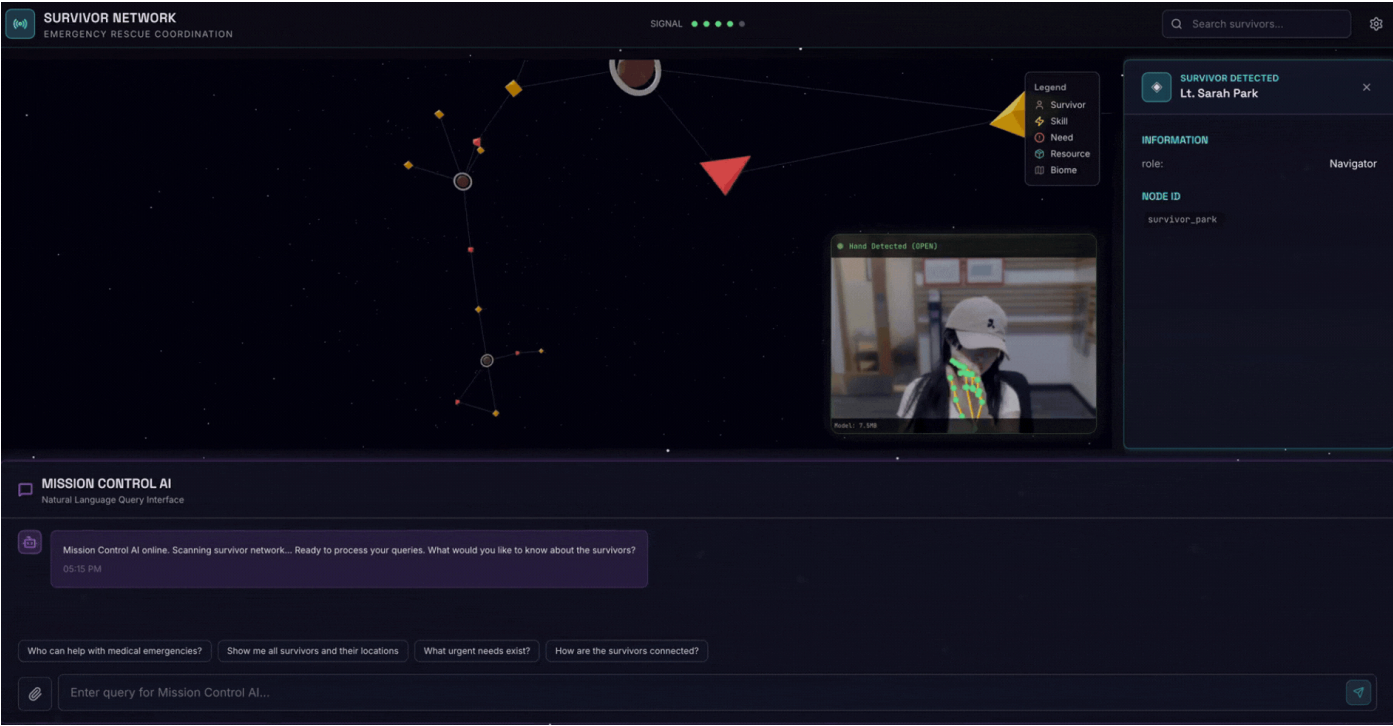
1. 簡介



1. 挑戰

在災害應變情境中，協調不同地點的生還者 (他們擁有不同技能、資源和需求) 需要智慧資料管理和搜尋功能。本研討會將教導您如何建構結合下列項目的 AI 生產系統：

1. 🗃️ **圖形資料庫 (Spanner)**：儲存倖存者、技能和資源之間的複雜關係
2. 🔍 **AI 輔助搜尋**：使用嵌入的語意 + 關鍵字混合搜尋
3. 🗂️ **多模態處理**：從圖片、文字和影片中擷取結構化資料
4. 🤖 **多代理自動化調度管理**：協調專門的代理，處理複雜的工作流程
5. 🧠 **長期記憶**：使用 Vertex AI Memory Bank 進行個人化



2. 建構項目

倖存者網路圖形資料庫，內含：

- 🗺️ 3D 互動式圖表，呈現倖存者之間的關係
- 🔍 智慧搜尋 (關鍵字、語意和混合)
- 📺 多模態上傳管道 (從圖片/影片中擷取實體)
- 🤖 多代理系統，可自動調度管理複雜工作
- 🧠 整合記憶體庫，提供個人化互動體驗

3. 核心技術

元件	科技	目的
資料庫	Cloud Spanner Graph	儲存節點 (倖存者、技能) 和邊緣 (關係)
AI 搜尋	Gemini + Embeddings	語意理解 + 相似度搜尋
代理程式架構	ADK (Agent Development Kit)	自動調度管理 AI 工作流程
記憶體	Vertex AI Memory Bank	長期儲存使用者偏好設定
前端	React + Three.js	互動式 3D 圖表視覺化

步驟 2 · 約 0 分鐘

2. 🛠️ 準備環境 (如果您在研討會中，請略過此步驟)

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

如果您是 **Google WayBackHome Workshop** 的與會者，且已完成第 0 級和第 1 級，請略過本課程！

第一部分：啟用帳單帳戶

如要執行本程式碼研究室，您需要有具備部分抵免額的帳單帳戶。請使用本程式碼研究室頂端橫幅中的抵免額開始操作。如果已連結帳單帳戶，可以略過這個步驟。

第二部分：開放式環境

1. 🖱️ 按一下這個連結，直接前往 [Cloud Shell 編輯器](#)
2. 🖱️ 如果系統在今天任何時間提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

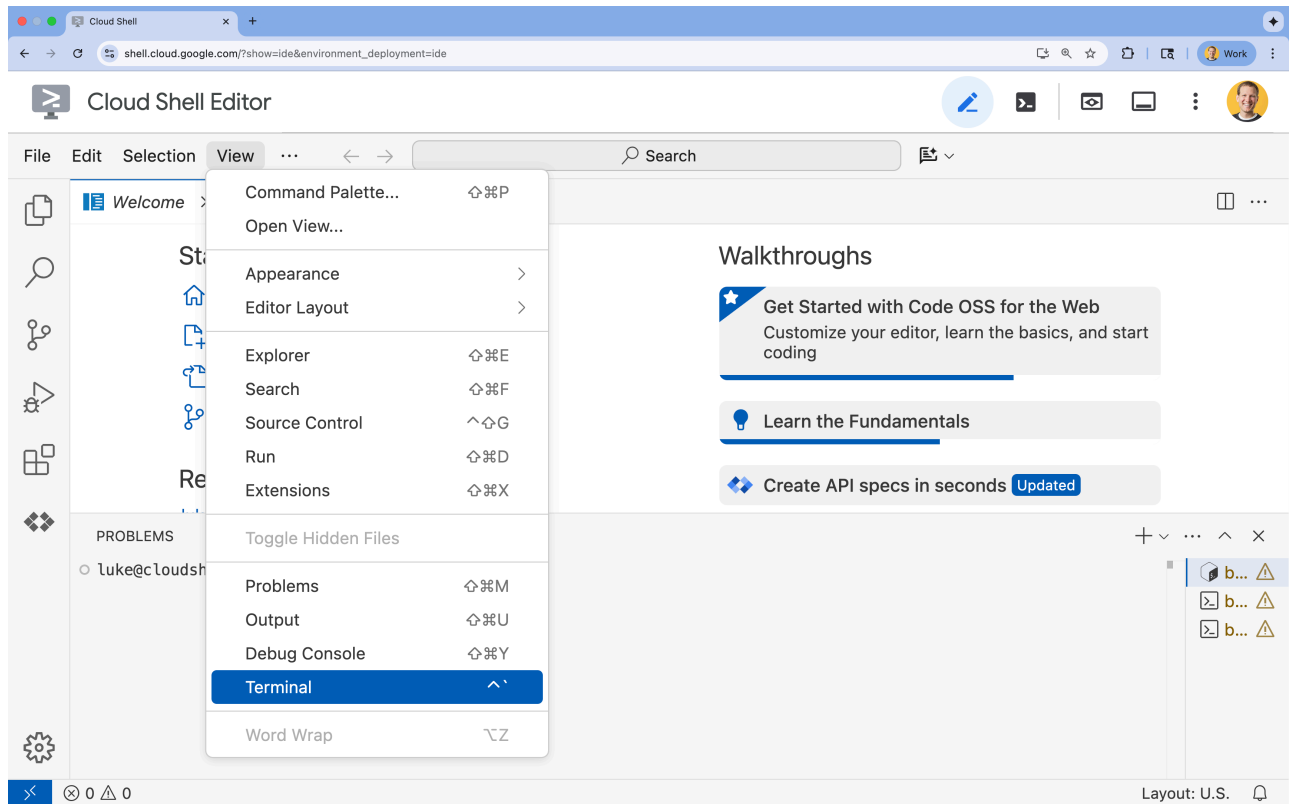
gcloud is requesting your credentials to make a GCP API call.

Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

3. 🖱️ 如果畫面底部未顯示終端機，請開啟終端機：
 - 按一下「查看」
 - 按一下「終端機」



4. 🖱️ 在終端機中，使用下列指令驗證您是否已通過驗證，以及專案是否已設為您的專案 ID：

```
gcloud auth list
```

5. 🖱️ 從 GitHub 複製啟動程序專案：

```
git clone https://github.com/gca-americas/way-back-home.git
```

第三部分：建立新專案

- 🖱️ 在終端機中，將 init 指令碼設為可執行檔並運作執行：

```
cd ~/way-back-home/level_2
./init.sh
```

⚠️ **專案 ID 注意事項：**指令碼會提示您建立新的 Google Cloud 專案。您可以接受系統隨機產生的預設專案 ID，也可以自行指定。

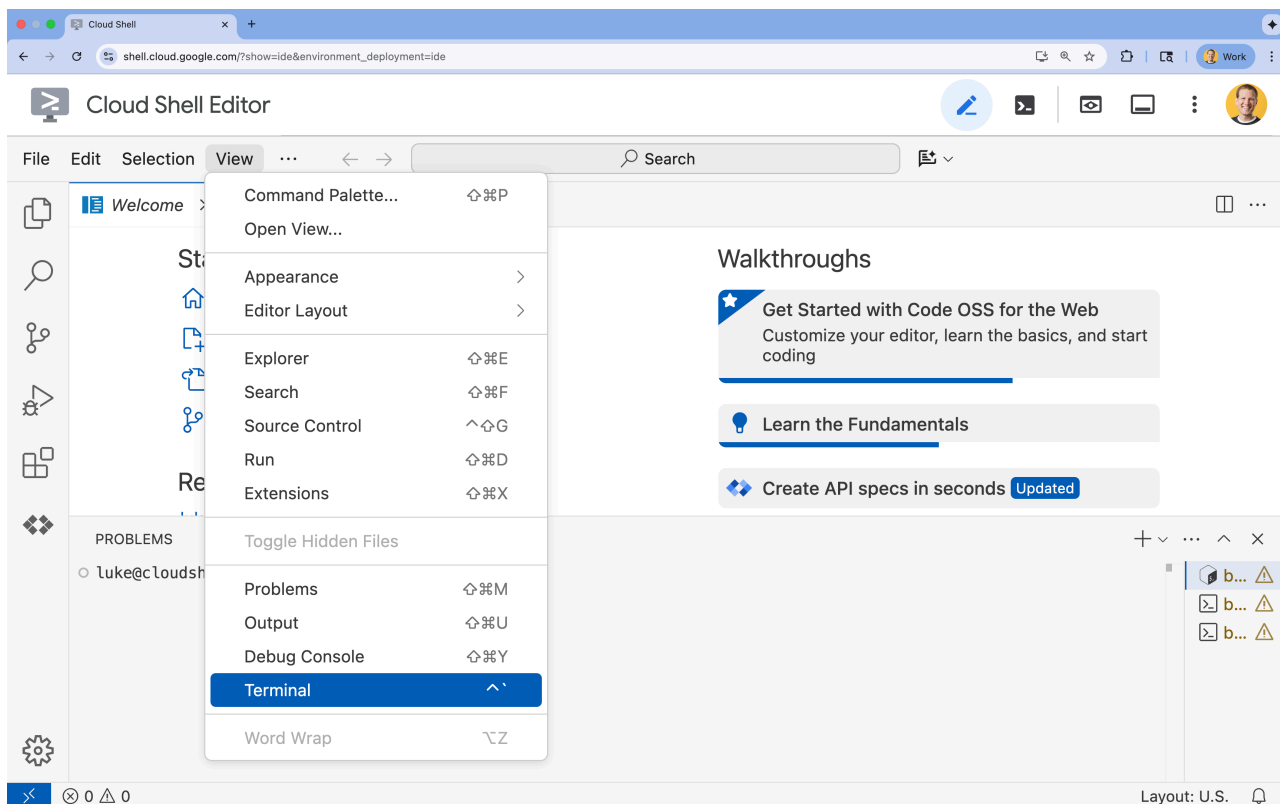
步驟 3 · 約 0 分鐘

3. 環境設定

1. 開啟 Cloud Shell

在 [Cloud Shell 編輯器](#) 終端機中，如果終端機未顯示在畫面底部，請開啟終端機：

- 按一下「查看」
- 按一下「終端機」。



如果您在工作坊中，並發現自己目前位於 `(.venv)`，請務必先執行 `deactivate` 再繼續。

2. 設定專案

👉 在終端機中設定專案 ID：

```
gcloud config set project $(cat ~/project_id.txt) --quiet
```

👉 啟用必要的 API (這需要約 2 到 3 分鐘)：

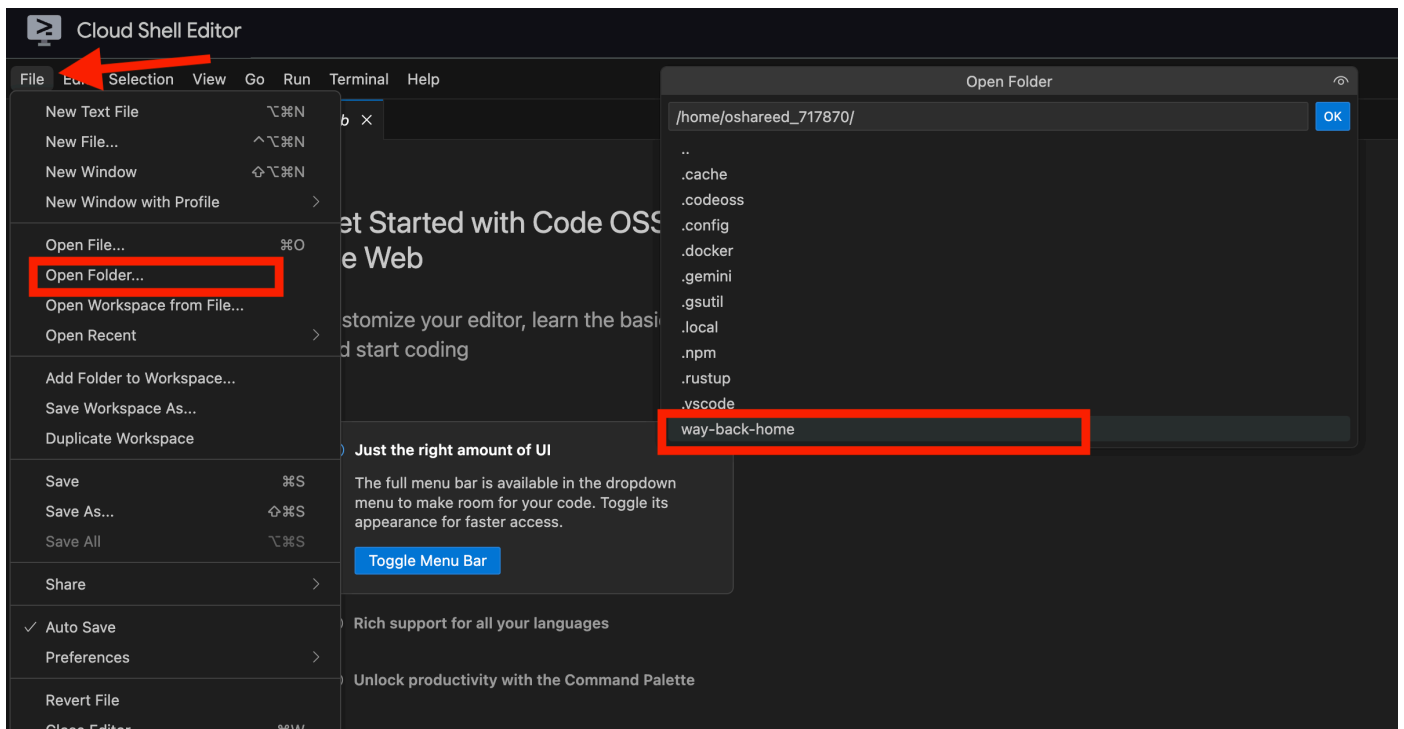
```
gcloud services enable compute.googleapis.com \  
    aiplatform.googleapis.com \  
    run.googleapis.com \  
    cloudbuild.googleapis.com \  
    artifactregistry.googleapis.com \  
    spanner.googleapis.com \  
    storage.googleapis.com
```

3. 執行設定指令碼

👉 執行設定指令碼：

```
cd ~/way-back-home/level_2
./setup.sh
```

系統會為您建立 `.env`。在 Cloud Shell 中開啟 `way_back_home` project. 在 `level_2` 資料夾下方，您會看到系統為您建立的 `.env` 檔案。如果找不到，請按一下 `View -> Toggle Hidden File` 即可查看。



4. 載入範例資料

👉 前往後端並安裝依附元件：

```
cd ~/way-back-home/level_2/backend
uv sync
```

👉 載入初始倖存者資料：

```
uv run python ~/way-back-home/level_2/backend/setup_data.py
```

這會建立：

- Spanner 執行個體 (`survivor-network`)
- 資料庫 (`graph-db`)
- 所有節點和邊緣資料表
- 用於查詢的屬性圖：預期的輸出內容：

```
=====
SUCCESS! Database setup complete.
=====
```

```
Instance: survivor-network
Database: graph-db
Graph: SurvivorGraph
```

```
Access your database at:
https://console.cloud.google.com/spanner/instances/survivor-network/databases/graph-db?
project=waybackhome
```

按一下輸出內容中 `Access your database at` 後方的連結，即可開啟 Google Cloud 控制台 Spanner。

```
=====
SUCCESS! Database setup complete.
=====

Instance: survivor-network
Database: graph-db
Graph: SurvivorGraph

Access your database at:
https://console.cloud.google.com/spanner/instances/survivor-network/databases/graph-db?project=waybackhome-4dni77umfbjrd8514w
```



您會在 Google Cloud 控制台看到 **Spanner** !

Google Cloud

waybackhome-4dni77umfbjrd8514w

Search (/) for resources, docs, products, and more

Search

Spanner

☆

All instances > Instance survivor-network: Overview > Google Standard SQL Database graph-db: Overview

Write DDL Delete database

DATABASE

Overview

Spanner Studio

Import/Export

Backup/Restore

Operations

Change streams

OBSERVABILITY

System insights

Query insights

Lock insights

Transaction insights

Hotspot insights

Key Visualizer

Introducing Spanner Vertex AI integration

Get seamless integration of ML predictions on Spanner. Use the Spanner Vertex AI integration to access classifier and regression ML models hosted on Vertex AI through the GoogleSQL interface.

Enable Learn More

Overview

Database Name	graph-db
Database Dialect	Google Standard SQL
Encryption type	Google-managed
Scheduled backups	default_daily_full_backup_schedule
Schema updates	No recent updates

Summary

Statistics are updated every 3-5 minutes, which may cause some delay in actual data.

CPU utilization (mean)	Operations	Throughput	Total database storage
—	Read: —/s	Read: —/s	—
	Write: 0.03/s	Write: 96 B/s	

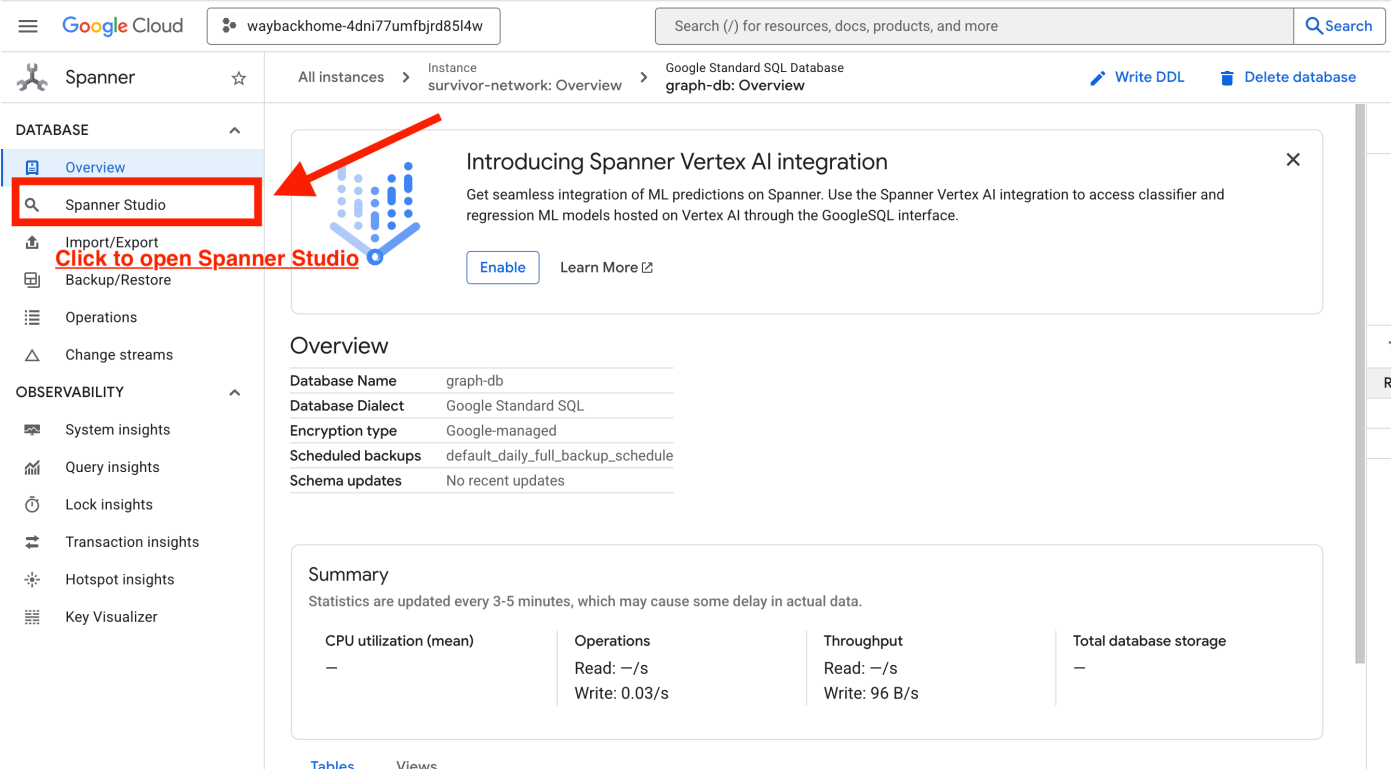
Tables Views

4. 🚀 在 Spanner Studio 中以圖表呈現圖形資料

本指南說明如何使用 Spanner Studio，直接在 Google Cloud 控制台中以視覺化方式呈現 Survivor Network 圖表資料，並與這些資料互動。在建構 AI 代理程式之前，這是驗證資料和瞭解圖表結構的好方法。

1. 存取 Spanner Studio

1. 在最後一個步驟中，請務必點選連結並開啟 Spanner Studio。



2. 瞭解圖結構 (「大方向」)

Survivor Network 資料集就像是邏輯謎題或遊戲狀態：

實體	系統中的角色	比喻
倖存者	代理程式/玩家	玩家
生物群系	所在位置	地圖區域
技能	可執行的操作	功能
需求	他們缺乏 (危機)	任務
資源	在現實世界中找到的項目	戰利品

目標：AI 代理程式的工作是將技能 (解決方案) 連結至需求 (問題)，同時考量生物群系 (地點限制)。

🔗 邊緣 (關係)：

- `SurvivorInBiome`：位置追蹤
- `SurvivorHasSkill`：能力清單
- `SurvivorHasNeed`：有效問題清單
- `SurvivorFoundResource`：項目清單
- `SurvivorCanHelp`：推斷關係 (AI 會計算這個！)

3. 查詢圖形

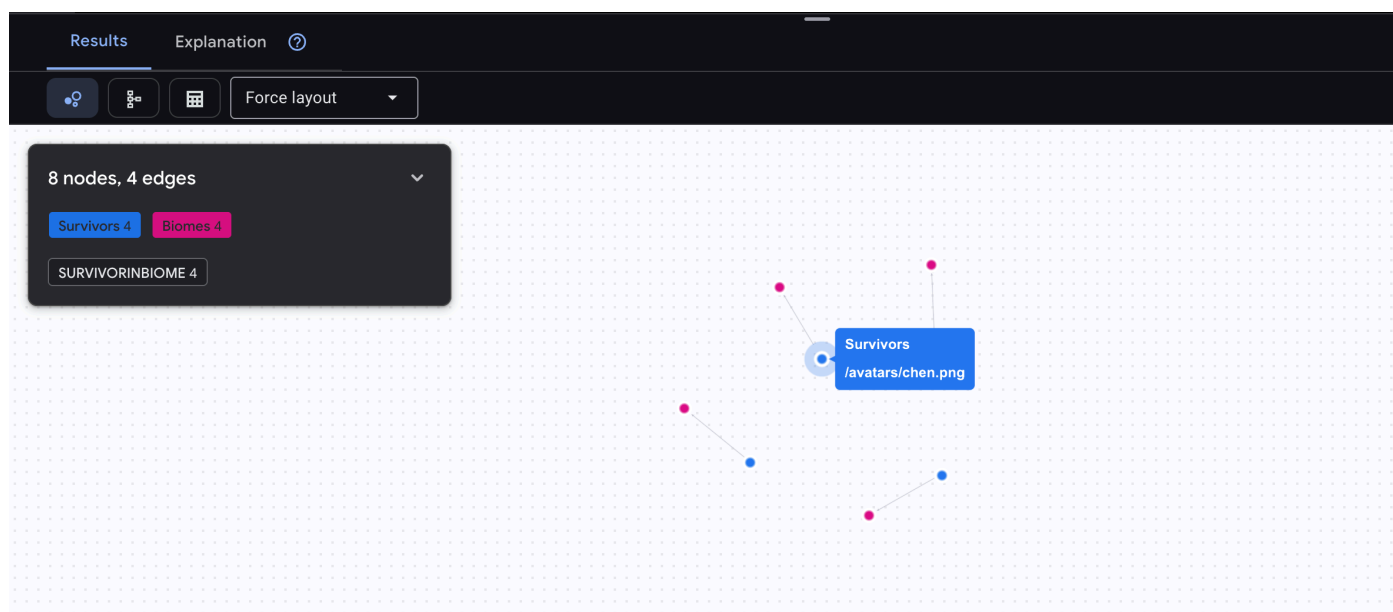
我們來執行幾項查詢，看看資料中的「Story」。

Spanner Graph 使用 **GQL (Graph Query Language)**。如要執行查詢，請使用 `GRAPH SurvivorNetwork`，然後輸入比對模式。

👉 **查詢 1：全球名單 (誰在哪裡?)** 這是基礎知識，瞭解位置資訊對救援行動至關重要。

```
GRAPH SurvivorNetwork
MATCH result = (s:Survivors)-[:SurvivorInBiome]->(b:Biomes)
RETURN TO_JSON(result) AS json_result
```

預期會看到如下結果：



重要性：

- **調度員**：你無法派火山區的人員前往冰寒區救援。
- **AI 代理**：當使用者詢問「森林裡有誰？」，這就是代理使用的查詢模式。
- **視覺化**：您會看到 4 名倖存者分布在 4 個生物群系中，這就是「地圖檢視畫面」。

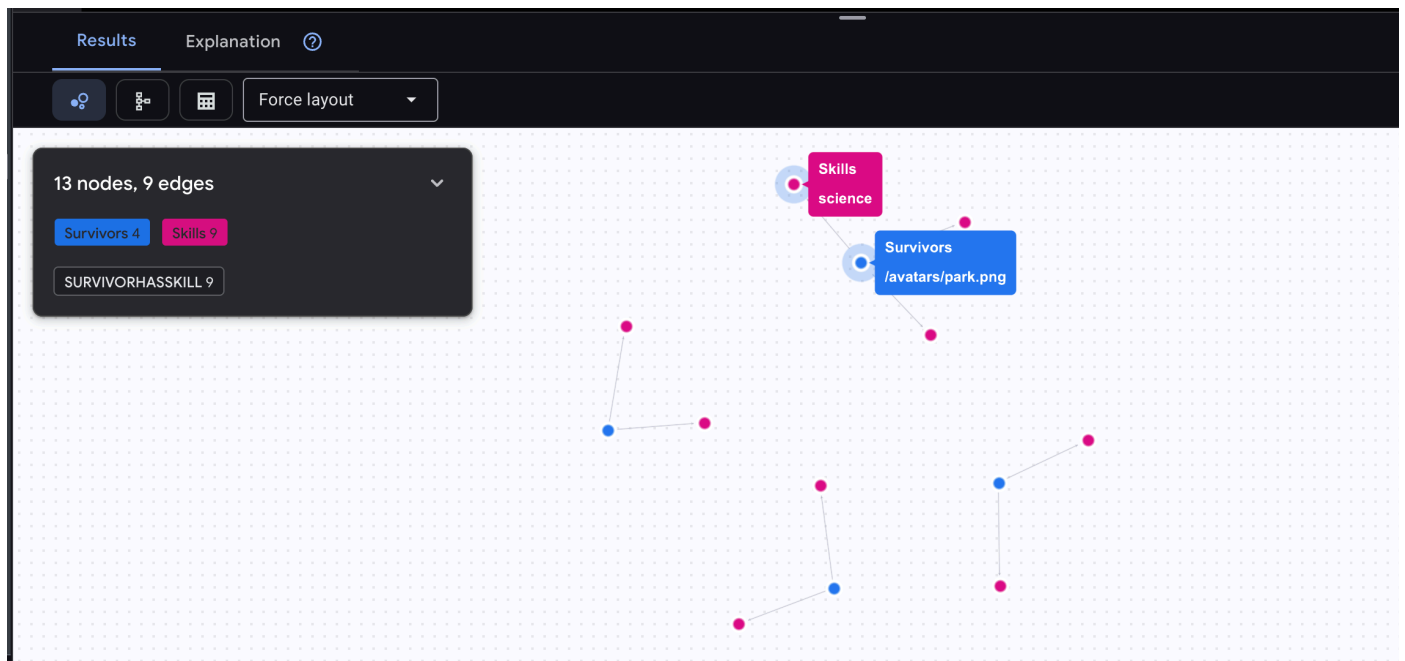
👉 **查詢 2：技能矩陣 (能力)** 瞭解每個人的所在地點後，請找出他們能做的事。

```

GRAPH SurvivorNetwork
MATCH result = (s:Survivors)-[h:SurvivorHasSkill]->(k:Skills)
RETURN TO_JSON(result) AS json_result

```

預期會看到如下結果：



重要性：

- 適用於調度員：一目瞭然地識別醫護人員、工程師和導航員。
- AI 代理：使用者詢問「誰會急救？」時，系統會找到他們。
- 熟練程度：請注意，邊緣 h 具有 proficiency 屬性 (專家/熟練/基本)，圖表會儲存關係中繼資料！

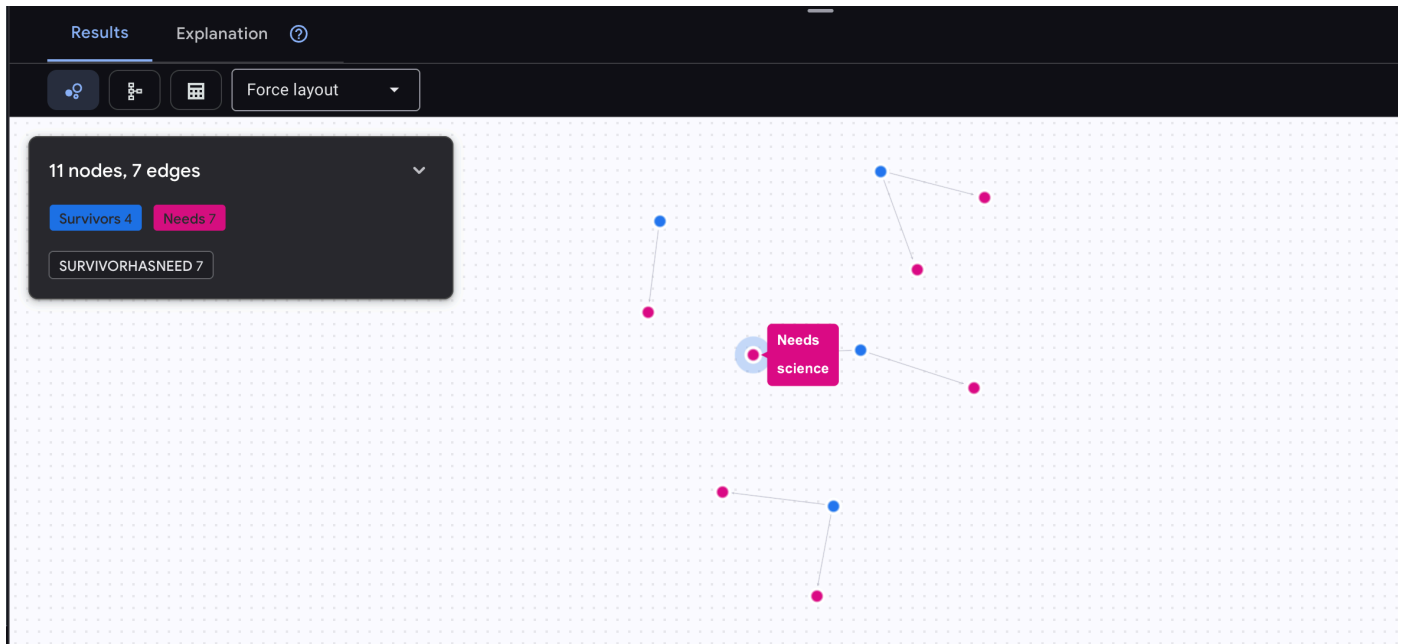
👉 查詢 3：誰處於危機狀態？(「任務板」) 查看需要協助的倖存者和他們的需求。

```

GRAPH SurvivorNetwork
MATCH result = (s:Survivors)-[h:SurvivorHasNeed]->(n:Needs)
RETURN TO_JSON(result) AS json_result

```

預期會看到如下結果：



重要性：

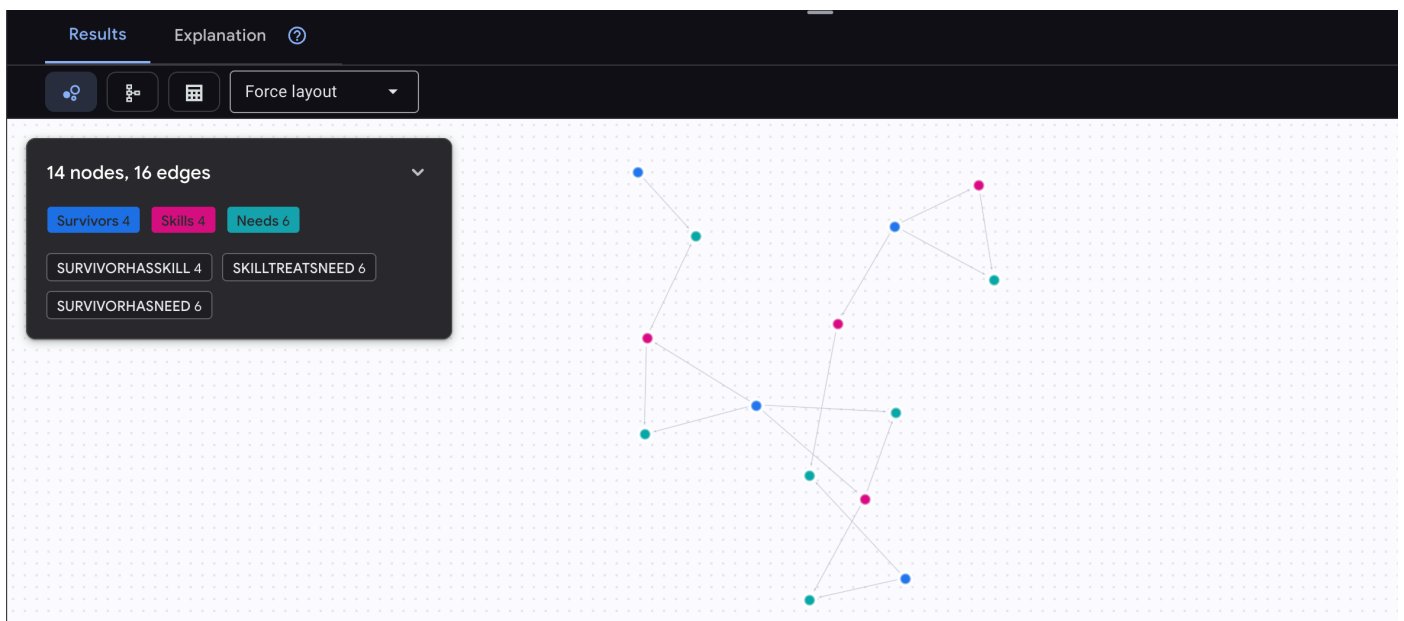
- 調度員：依緊急程度安排救援任務的優先順序。
- AI 代理程式：這是「待辦事項清單」，列出待解決的問題。

🔍 [選用] 媒合 - 誰可以協助誰？

這時圖表就能發揮強大功能！這項查詢會找出具備可滿足其他倖存者需求的技能的倖存者。

```
GRAPH SurvivorNetwork
MATCH result = (helper:Survivors)-[:SurvivorHasSkill]->(skill:Skills)-[:SkillTreatsNeed]->
(needs:Needs)<-[:SurvivorHasNeed]-(helpee:Survivors)
RETURN TO_JSON(result) AS json_result
```

預期會看到如下結果：



aside positive 這項查詢的作用：

這項查詢不會只顯示「急救可治療燒傷」(這從結構化資料中顯而易見)，而是會找出：

- **Elena Frost 醫生** (接受過醫療訓練) → 可以治療 → **田中隊長** (有燒傷)
- **陳大衛** (具備急救技能) → 可以治療 → **朴中尉** (腳踝扭傷)

這項功能為何如此強大：

圖表優勢：這是 4 個躍點的遍歷：

1. 從「幫手」(倖存者) 開始
2. 查看他們的技能
3. 將這些技能與可解決的需求配對
4. 找出有這些需求的倖存者

如果沒有圖表，您需要：

- 查詢 SurvivorHasSkill 資料表 → 取得技能
- 查詢 SkillTreatsNeed 資料表 → 找出相符需求
- 查詢 SurvivorHasNeed 資料表 → 找出有這些需求的人
- 在應用程式程式碼中加入所有結果

使用 Spanner Graph 時，資料庫會為您最佳化單一查詢。

AI 代理會執行的動作：

當使用者詢問「誰可以治療燒燙傷？」時，服務專員會：

1. 執行類似的圖表查詢
 2. 回覆：「Dr. Frost has Medical Training and can help Captain Tanaka」(Frost 醫生接受過醫療訓練，可以協助田中隊長)
 3. 使用者不必瞭解中介資料表或關係！
-

5. 🚀 Spanner 中的 AI 輔助嵌入

1. 為什麼要使用嵌入項目？(不執行任何動作，唯讀)

在求生情境中，時間至關重要。當倖存者回報緊急狀況 (例如 `I need someone who can treat burns` 或 `Looking for a medic`) 時，他們無法浪費時間猜測資料庫中的確切技能名稱。

傳統搜尋的缺點：

- **完全比對**：如果關鍵字搜尋「醫護人員」，就不會找到「醫療訓練」或「急救」
- **詞彙不符**：倖存者可能會說「醫生」、「治療師」、「醫護人員」，但資料庫儲存的是正式技能名稱
- **損失**：「處理燒傷」應找出「醫療訓練」和「急救」，但簡單的文字搜尋無法瞭解語意關係

解決方案：使用嵌入的語意搜尋：嵌入會將文字轉換為 **768 維度的向量**，並從中捕捉意義，而不只是拼字。相似概念會在向量空間中聚在一起，因此即使沒有任何字詞重疊，「醫護人員」自然會找到「醫療訓練」。

實際情境：倖存者：`Captain Tanaka has burns—we need medical help NOW!`

傳統關鍵字搜尋「醫護人員」→ 0 筆結果 ❌

使用嵌入的語意搜尋 → 找到「醫療訓練」、「急救」✅

這正是代理程式需要的：**智慧型搜尋**，類似人類的搜尋方式，可瞭解意圖，而不只是關鍵字。

為何選擇 Spanner？：您不必將資料匯出至 Python、在外部產生嵌入內容，然後重新匯入 (速度緩慢且容易出錯)，Spanner 的 **ML.PREDICT** 可讓您：

- 使用 Vertex AI 模型**直接在 SQL 中**生成嵌入
- **與圖形資料一起儲存向量** (不需另外建立向量資料庫)
- 在圖形遍歷期間即時執行**語意搜尋查詢**

這會建立強大的圖表 **RAG** 系統：圖表關係 (誰具備哪些技能) + 語意搜尋 (依據意義尋找技能) = 智慧型代理程式查詢。

2. 建立嵌入模型

什麼是 ML.PREDICT ?

`ML.PREDICT` 是 Spanner 的內建函式，可讓您**直接從 SQL 呼叫機器學習模型**，不必使用 Python、匯出/匯入，也不需要個別服務。這項功能可視為即時連結資料庫和 Vertex AI。

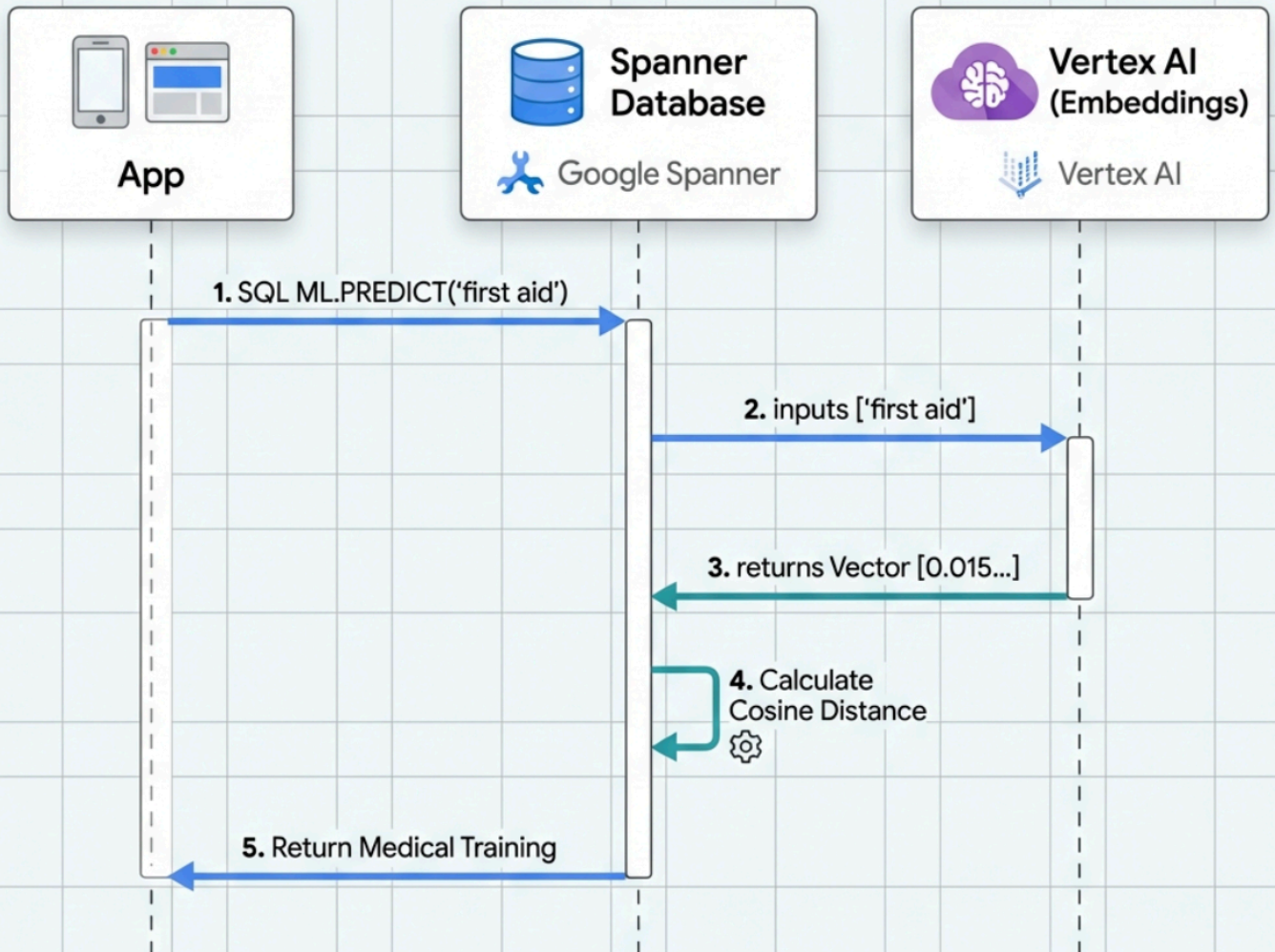
運作方式：

1. 在 Spanner 中建立**虛擬模型** (僅供參考，不會在本機儲存模型權重)
2. 在查詢中使用 `ML.PREDICT(model, data)` 時，Spanner 會透過 API 將資料傳送至 Vertex AI
3. Vertex AI 會處理這項要求，並在 SQL 結果中以**資料欄形式**傳回結果

強大之處：

-  **零 ETL**：在資料所在位置處理資料，不必匯出至 Python 指令碼
-  **即時**：在查詢期間即時生成嵌入項目或 LLM 回覆
-  **可擴充**：Spanner 負責自動化調度管理，Vertex AI 負責運算

Spanner Embedding Flow

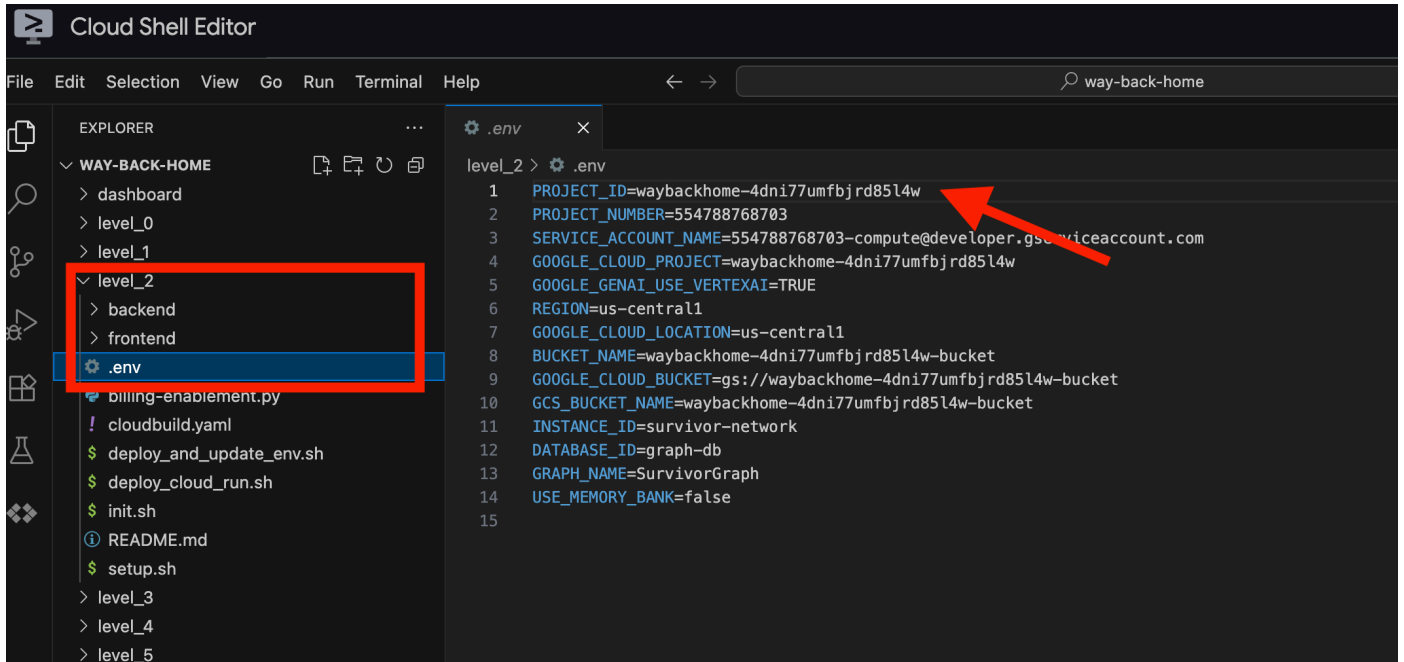


現在，讓我們使用 Google 的 `text-embedding-004` 建立模型，將文字轉換為嵌入。

👉 在 Spanner Studio 中執行這項 SQL (將 `$YOUR_PROJECT_ID` 換成實際專案 ID)：

請務必將 `$YOUR_PROJECT_ID` 換成實際專案 ID，您可以在 `.env` 檔案中找到專案 ID

!! 在 Cloud Shell 編輯器中，開啟 `File -> Open Folder -> way-back-home/level_2`，即可查看整個專案。



👉 在 Spanner Studio 中執行這項查詢：複製並貼上下列查詢，然後點選「執行」按鈕：

```
CREATE OR REPLACE MODEL TextEmbeddings
INPUT(content STRING(MAX))
OUTPUT(embeddings STRUCT<values ARRAY<FLOAT32>>)
REMOTE OPTIONS (
  endpoint = '//aiplatform.googleapis.com/projects/$YOUR_PROJECT_ID/locations/us-
centrall1/publishers/google/models/text-embedding-004'
);
```

如果忘記取代 `$YOUR_PROJECT_ID` 並已執行，請務必重新執行。

用途：

- 在 Spanner 中建立虛擬模型 (不會在本機儲存模型權重)
- Vertex AI 的 `text-embedding-004`
- 定義合約：輸入為文字，輸出為 768 維度的浮點數陣列

為什麼是「遠端選項」？

- Spanner 本身不會執行模型
- 使用 `ML.PREDICT` 時，系統會透過 API 呼叫 Vertex AI
- 零 ETL：不必將資料匯出至 Python、處理及重新匯入

按一下 `Run` 按鈕，成功後即可看到如下所示的結果：

Untitled query • Untitled query • Untitled query •

Open CLI Gemini settings ▾

Run ▾

Save ▾

Format

Clear

Documentation

✓

```
1 CREATE MODEL TextEmbeddings
2 INPUT(content STRING(MAX))
3 OUTPUT(embeddings STRUCT<values ARRAY<FLOAT32>>)
4 REMOTE OPTIONS (
5   | endpoint = '//aiplatform.googleapis.com/projects/waybackhome-4dni77umfbjrd8514w/locations/us-central1/publishers/google/models/text-embedding-004'
6 );
```

replace \$project_id with real project id

Results

Explanation

✓ Update completed
To view more details, go to [Operations Page](#).
To view all updates go to [Logging](#).

這會建立參照 Vertex AI text-embedding-004 的「虛擬模型」。資料不會離開 Spanner，而是透過 API 呼叫。

3. 新增嵌入欄

👉 新增資料欄來儲存嵌入內容：

```
ALTER TABLE Skills ADD COLUMN skill_embedding ARRAY<FLOAT32>;
```

按一下 `Run` 按鈕，成功後即可看到如下所示的結果：

Run ▾

Save ▾

Format

Clear

Documentation

✓

```
1 ALTER TABLE Skills ADD COLUMN skill_embedding ARRAY<FLOAT32>;
```

Results

Explanation

✓

Update completed
To view more details, go to [Operations Page](#).
To view all updates go to [Logging](#).

4. 生成嵌入

👉 使用 AI 為每項技能建立向量嵌入：

```
UPDATE Skills
SET skill_embedding = (
  SELECT embeddings.values
  FROM ML.PREDICT(
    MODEL TextEmbeddings,
    (SELECT name AS content)
  )
)
WHERE skill_embedding IS NULL;
```

按一下 **Run** 按鈕，成功後即可看到如下所示的結果：

✓

```
1 UPDATE Skills
2 SET skill_embedding = (
3   SELECT embeddings.values
4   FROM ML.PREDICT(
5     MODEL TextEmbeddings,
6     (SELECT name AS content)
7   )
8 )
9 WHERE skill_embedding IS NULL;
```

Results

Explanation ?

10 rows updated

This statement updated 10 rows and did not return any rows.

如果看到權限錯誤，請重新執行步驟 2「建立嵌入模型」。請務必先執行 `DROP MODEL TextEmbeddings;`，再重試。

發生情況：每個技能名稱 (例如「急救」) 都會轉換為代表語意含義的 768 維度向量。

5. 驗證嵌入

👉 確認是否已建立嵌入：

```
SELECT
    skill_id,
    name,
    ARRAY_LENGTH(skill_embedding) AS embedding_dimensions
FROM Skills
LIMIT 5;
```

預期的輸出內容：

Run Save Format Clear Documentation

✓

```
1 SELECT
2     skill_id,
3     name,
4     ARRAY_LENGTH(skill_embedding) AS embedding_dimensions
5 FROM Skills
6 LIMIT 5;
```

Results Explanation ?

skill_id	name	embedding_dimensions
"skill_botany"	"Botany"	768
"skill_cartography"	"Cartography"	768
"skill_engineering"	"Engineering"	768
"skill_first_aid"	"First Aid"	768
"skill_foraging"	"Foraging"	768

6. 測試語意搜尋

現在來測試情境中的確切用途：使用「醫護人員」一詞尋找醫療技能。

👉 尋找與「醫護人員」類似的技能：

```
WITH query_embedding AS (  
  SELECT embeddings.values AS val  
  FROM ML.PREDICT(MODEL TextEmbeddings, (SELECT "medic" AS content))  
)  
SELECT  
  s.name AS skill_name,  
  s.category,  
  COSINE_DISTANCE(s.skill_embedding, (SELECT val FROM query_embedding)) AS distance  
FROM Skills AS s  
WHERE s.skill_embedding IS NOT NULL  
ORDER BY distance ASC  
LIMIT 10;
```

- 將使用者的搜尋字詞「醫生」轉換為嵌入
- 儲存在 query_embedding 臨時資料表

預期結果 (距離越小表示越相似)：

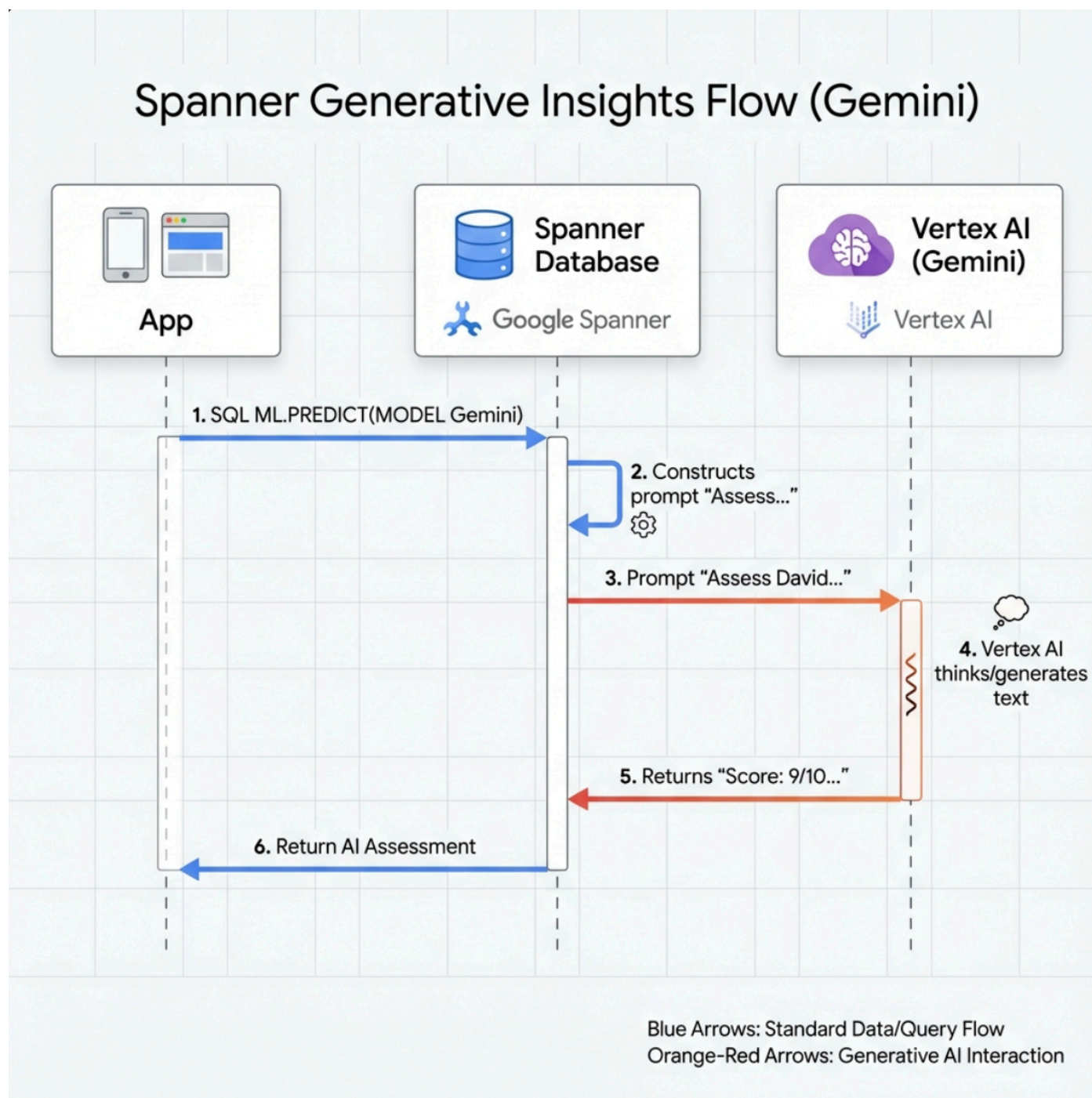
Run Save Format Clear Documentation

1 WITH query_embedding AS (
2 | SELECT embeddings.values AS val
3 | FROM ML.PREDICT(MODEL TextEmbeddings, (SELECT "medic" AS content))
4 |)
5 SELECT
6 | s.name AS skill_name,
7 | s.category,
8 | COSINE_DISTANCE(s.skill_embedding, (SELECT val FROM query_embedding)) AS distance
9 FROM Skills AS s
10 WHERE s.skill_embedding IS NOT NULL
11 ORDER BY distance ASC
12 LIMIT 10;

Results Explanation

skill_name	category	distance
"Medical Training"	"medical"	0.37703354826388813
"First Aid"	"medical"	0.39846303316405174
"Pilot"	"technical"	0.6023317061249763
"Botany"	"science"	0.6088331602164481
"Engineering"	"technical"	0.6266319727618473
"Navigation"	"technical"	0.6673448489010516
"Cartography"	"science"	0.6846843147059241
"Leadership"	"leadership"	0.6982514482900226
"Xenobiology"	"science"	0.7108070853551869
"Foraging"	"survival"	0.7166774377313498

7. 建立 Gemini 模型以進行分析



👉 建立生成式 AI 模型參照 (將 `$YOUR_PROJECT_ID` 換成實際專案 ID) :

請務必將 `$YOUR_PROJECT_ID` 換成實際專案 ID，您可以在 `.env` 檔案中找到專案 ID

```
CREATE OR REPLACE MODEL GeminiPro
INPUT(prompt STRING(MAX))
OUTPUT(content STRING(MAX))
REMOTE OPTIONS (
    endpoint = '//aiplatform.googleapis.com/projects/$YOUR_PROJECT_ID/locations/us-central1/publishers/google/models/gemini-2.5-pro',
    default_batch_size = 1
);
```

如果忘記取代 `$YOUR_PROJECT_ID` 並已執行，請務必重新執行。

與嵌入模型不同之處：

- 嵌入：文字 → 向量 (用於相似度搜尋)
- **Gemini**：文字 → 生成文字 (用於推論/分析)

Run

Save

Format

Clear

Documentation

1 CREATE MODEL GeminiPro

2 INPUT(prompt STRING(MAX))

3 OUTPUT(content STRING(MAX))

4 REMOTE OPTIONS (

5 | endpoint = '//aiplatform.googleapis.com/projects/waybackhome-4dni77umfbjrd8514w/locations/us-central1/publishers/google/models/gemini-2.5-pro',

6 | default_batch_size = 1

7);

replace \$project_id with real project id

Results

Explanation

Update completed

To view more details, go to [Operations Page](#).

To view all updates go to [Logging](#).

8. 使用 Gemini 進行相容性分析

👉 分析倖存者配對的任務相容性：


```

WITH PairData AS (
    SELECT
        s1.name AS Name_A,
        s2.name AS Name_B,
        CONCAT(
            "Assess compatibility of these two survivors for a resource-gathering mission. ",
            "Survivor 1: ", s1.name, ". ",
            "Survivor 2: ", s2.name, ". ",
            "Give a score from 1-10 and a 1-sentence reason."
        ) AS prompt
    FROM Survivors s1
    JOIN Survivors s2 ON s1.survivor_id < s2.survivor_id
    LIMIT 1
)
SELECT
    Name_A,
    Name_B,
    content AS ai_assessment
FROM ML.PREDICT(
    MODEL GeminiPro,
    (SELECT Name_A, Name_B, prompt FROM PairData)
);

```

預期的輸出內容：

Name_A	Name_B	ai_assessment
"David Chen"	"Dr. Elena Frost"	"**Score: 9/10** Their compatibility is extremely high as David's practical, hands-on scavenging skills are perfectly complemented by Dr. Frost's specialized knowledge to identify critical medical supplies and avoid biological hazards."

運作方式：

1. 根據資料建立提示詞：

```
CONCAT("Assess compatibility...", "Survivor 1: ", s1.name, ...)
```

結果："Assess compatibility of these two survivors for a resource-gathering mission. Survivor 1: David Chen. Survivor 2: Dr. Elena Frost. Give a score from 1-10 and a 1-sentence reason."

2. 傳送到 Gemini：

```
ML.PREDICT(MODEL GeminiPro, (SELECT prompt FROM PairData))
```

Spanner 會將提示傳送至 Gemini API。

3. Gemini 生成回覆：

```
"""Score: 9/10** Their compatibility is extremely high as David's practical, hands-on scavenging skills are perfectly complemented by Dr. Frost's specialized knowledge to identify critical medical supplies and avoid biological hazards."""
```

4. 以資料欄形式傳回：生成的文字會以 content 資料欄的形式傳回。

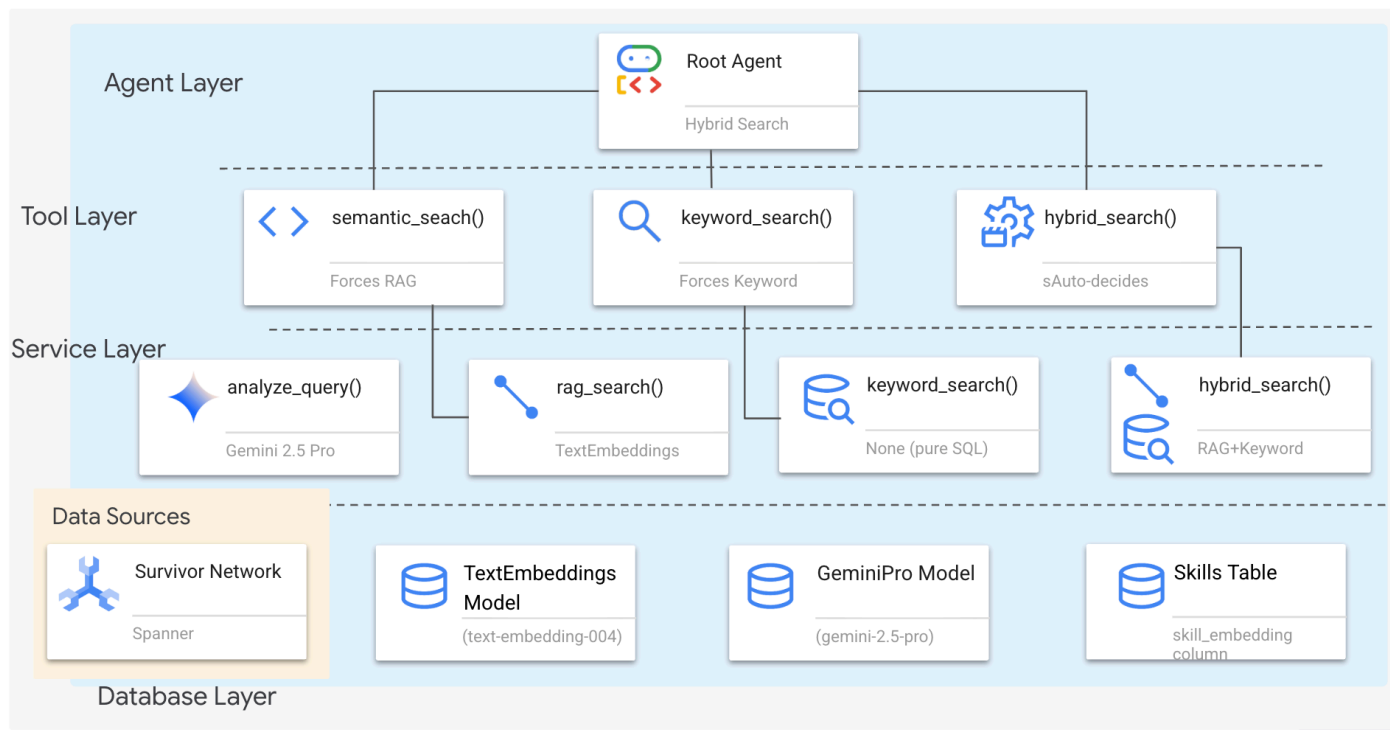
為什麼要使用資料庫內 LLM 呼叫？

- 不需要將資料匯出至 Python
- 在資料儲存位置處理資料
- 可擴充至數百萬列 (批次處理)

6. 🚀 使用混合搜尋建構 Graph RAG 代理程式

1. 系統架構總覽

本節將建構多種方法的搜尋系統，讓代理程式能彈性處理不同類型的查詢。這個系統有三層：代理程式層、工具層、服務層。



為什麼要分成三層？

- **關注點分離：**代理專注於意圖，工具專注於介面，服務專注於實作
- **彈性：**代理程式可以強制使用特定方法，也可以讓 AI 自動轉送
- **最佳化：**如果方法已知，可以略過昂貴的 AI 分析

在本節中，您主要會實作**語意搜尋 (RAG)**，也就是依據意義 (而非只有關鍵字) 查找結果。稍後，我們會說明**混合搜尋**如何合併多種方法。

2. RAG 服務實作

👉 在終端機中執行下列指令，在 Cloud Shell 編輯器中開啟檔案：

```
cloudshell edit ~/way-back-home/level_2/backend/services/hybrid_search_service.py
```

找出留言 # TODO: REPLACE_SQL

將整行程式碼替換為下列程式碼：

```
# This is your working query from the successful run!
sql = """
    WITH query_embedding AS (
        SELECT embeddings.values AS val
        FROM ML.PREDICT(
            MODEL TextEmbeddings,
            (SELECT @query AS content)
        )
    )
    SELECT
        s.survivor_id,
        s.name AS survivor_name,
        s.biome,
        sk.skill_id,
        sk.name AS skill_name,
        sk.category,
        COSINE_DISTANCE(
            sk.skill_embedding,
            (SELECT val FROM query_embedding)
        ) AS distance
    FROM Survivors s
    JOIN SurvivorHasSkill shs ON s.survivor_id = shs.survivor_id
    JOIN Skills sk ON shs.skill_id = sk.skill_id
    WHERE sk.skill_embedding IS NOT NULL
    ORDER BY distance ASC
    LIMIT @limit
    """
```

這項查詢的作用：

這是 **Graph RAG** 查詢，結合了兩項強大技術：

1. 圖表關係 (透過 Spanner 圖表資料表上的 SQL JOIN)
2. 語意搜尋 (透過嵌入和 `COSINE_DISTANCE`)

等等，這只是 **SQL** 嗎？

當然可以！Spanner Graph 讓您可以使用 **標準 SQL** 和 `JOIN` 陳述式，查詢圖形關係。圖表結構 (透過 `SurvivorHasSkill` 邊緣的倖存者 → 技能) 會儲存在您可以彙整的關聯式資料表中。這與需要 Cypher 或 Gremlin 等專用查詢語言的傳統圖形資料庫不同。

為什麼這是「圖形 RAG」：

- **圖表**：我們正在遍歷圖表結構中儲存的關係 (倖存者 → 技能)
- **RAG**：我們使用嵌入功能擷取語意相似的內容 (檢索增強生成)
- **結果**：根據技能的意義，透過圖形連結尋找倖存者

範例：查詢「醫護人員」→ 找出具備「醫療訓練」或「急救」技能的倖存者，即使資料庫中沒有「醫護人員」一詞也沒關係。

3. 語意搜尋工具定義

👉 在終端機中執行下列指令，在 Cloud Shell 編輯器中開啟檔案：

```
cloudshell edit ~/way-back-home/level_2/backend/agent/tools/hybrid_search_tools.py
```

在 `hybrid_search_tools.py` 中找到註解 `# TODO: REPLACE_SEMANTIC_SEARCH_TOOL`

👉 將整行程式碼替換為下列程式碼：

```

async def semantic_search(query: str, limit: int = 10) -> str:
    """
    Force semantic (RAG) search using embeddings.

    Use this when you specifically want to find things by MEANING,
    not just matching keywords. Great for:
    - Finding conceptually similar items
    - Handling vague or abstract queries
    - When exact terms are unknown

    Example: "healing abilities" will find "first aid", "surgery",
    "herbalism" even though no keywords match exactly.

    Args:
        query: What you're looking for (describe the concept)
        limit: Maximum results

    Returns:
        Semantically similar results ranked by relevance
    """
    try:
        service = _get_service()
        result = service.smart_search(
            query,
            force_method=SearchMethod.RAG,
            limit=limit
        )

        return _format_results(
            result["results"],
            result["analysis"],
            show_analysis=True
        )

    except Exception as e:
        return f"Error in semantic search: {str(e)}"

```

代理的使用時機：

- 要求相似度的查詢 (「尋找與 X 相似的內容」)
- 概念查詢 (「療癒能力」)
- 需要瞭解確切含義時

4. 代理人決策指南 (說明)

在代理程式定義中，將語意搜尋相關部分複製並貼到指令中。

👉 在終端機中執行下列指令，在 Cloud Shell 編輯器中開啟檔案：

```
cloudshell edit ~/way-back-home/level_2/backend/agent/agent.py
```

代理會根據這項指令選取合適的工具：

👉 在 `agent.py` 檔案中，找出註解 `# TODO: REPLACE_SEARCH_LOGIC`，將整行程式碼替換為下列程式碼：

```
- `semantic_search`: Force RAG/embedding search
  Use for: "Find similar to X", conceptual queries, unknown terminology
  Example: "Find skills related to healing"
```

👉 找出註解 `# TODO: ADD_SEARCH_TOOL` **Replace this whole line**，並替換成下列程式碼：

```
semantic_search,          # Force RAG
```

5. 瞭解混合型搜尋的運作方式 (僅供參考，不需採取任何行動)

在步驟 2 到 4 中，您實作了語意搜尋 (RAG)，這是透過意義尋找結果的核心搜尋方法。但您可能已注意到，這個系統稱為「混合式搜尋」。整體運作流程如下：

三種可用的搜尋方法：

1. 語意搜尋 (RAG) - 您剛才實作的內容

- 使用嵌入 + `COSINE_DISTANCE`
- 適用於：概念查詢、尋找「與 X 類似」的內容
- 例如：「治療能力」會找出「急救」、「手術」、「草藥學」

2. 關鍵字搜尋 - 傳統 SQL 篩選

- 使用 `LIKE` 子句和完全比對類別
- 最適合：特定篩選器，例如生物群系、類別
- 例如：「森林中的醫療技能」會依類別和地點篩選

3. 混合型搜尋 - 結合兩種方法

- 同時執行兩種搜尋，並合併結果和加權分數
- 最佳用途：同時包含概念和篩選條件的複雜查詢
- 例如：「Who can help with healing in the mountains?」（誰能在山區提供醫療協助？）

混合合併的運作方式：

在 `way-back-home/level_2/backend/services/hybrid_search_service.py` 檔案中，當呼叫 `hybrid_search()` 時，服務會執行「兩項」搜尋並合併結果：

```
# Location: backend/services/hybrid_search_service.py

rank_kw = keyword_ranks.get(surv_id, float('inf'))
rank_rag = rag_ranks.get(surv_id, float('inf'))

rrf_score = 0.0
if rank_kw != float('inf'):
    rrf_score += 1.0 / (K + rank_kw)
if rank_rag != float('inf'):
    rrf_score += 1.0 / (K + rank_rag)

combined_score = rrf_score
```

呼叫 `hybrid_search()` 時，服務會執行這兩種搜尋，並使用倒數排名融合 (RRF) 合併結果。

什麼是 RRF ? RRF 是標準演算法，可合併不同搜尋引擎的排名清單，不必正規化分數。這項功能會根據排名位置 (第 1、2、3 名) 給分，而非原始相似度分數。

各方法的使用時機

代理 (`backend/agent/agent.py`) 會根據查詢內容做出決定：

查詢類型	代理程式選擇	原因
「Find survivors in forest」(在森林中尋找生還者)	<code>keyword_search</code>	簡單篩選器，不需要語意
「Who can fix injuries?」	<code>semantic_search</code>	概念性，需要意義
「Medical help in mountains」(山區醫療協助)	<code>hybrid_search</code>	同時有概念和篩選器

在本程式碼研究室中，您實作了語意搜尋元件 (RAG)，這是基礎。服務中已導入關鍵字和混合式方法，因此代理程式可以使用這三種方法！

恭喜！您已順利完成混合搜尋的 Graph RAG 代理程式！

步驟 7 · 約 0 分鐘

7. 🚀 使用 ADK Web 測試代理

測試代理最簡單的方法是使用 `adk web` 指令，這會啟動代理並顯示內建的即時通訊介面。

1. 執行代理程式

👉 前往後端目錄 (定義代理的位置)，然後啟動網頁介面：

```
cd ~/way-back-home/level_2/backend
uv run adk web
```

這個指令會啟動

```
agent/agent.py
```

，並開啟測試用的網頁介面。

👉 開啟網址：

這個指令會輸出本機網址 (通常是 `http://127.0.0.1:8000` 或類似網址)。在瀏覽器中開啟。

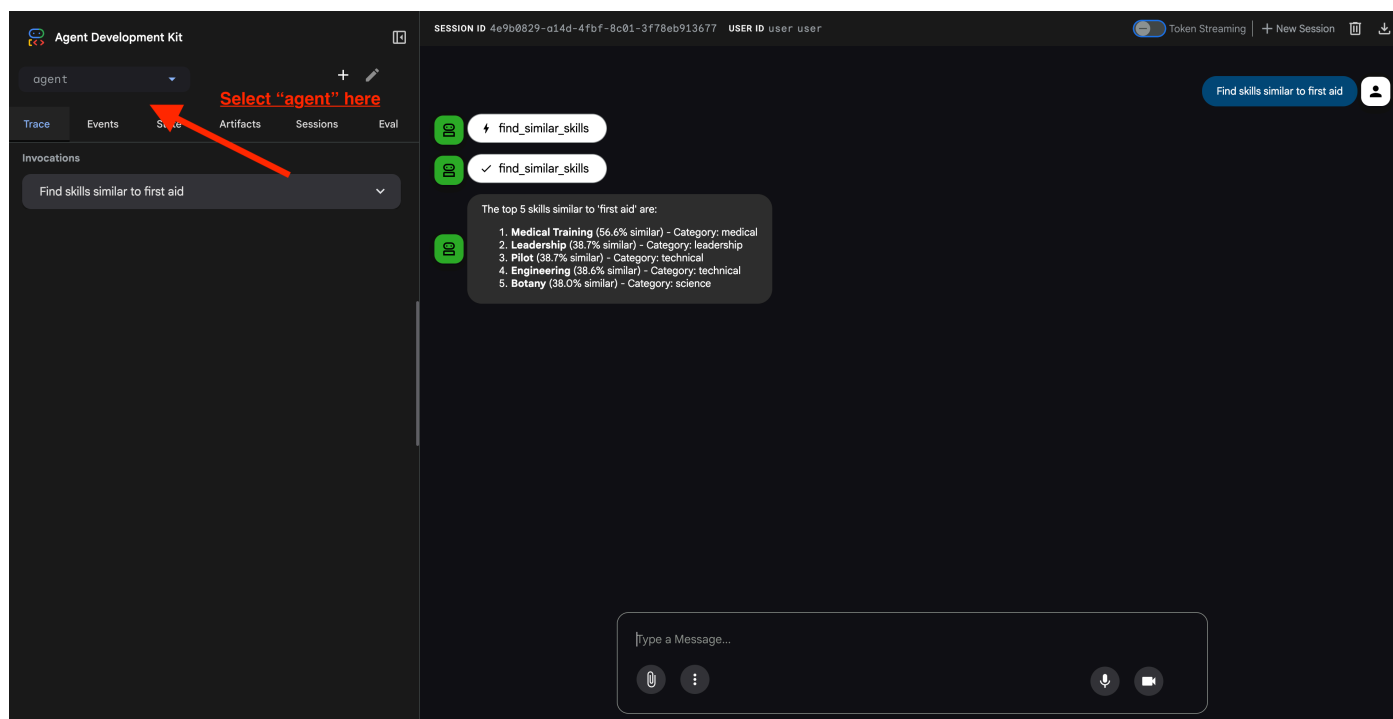
```
super().__init__()
INFO: Started server process [20529]
INFO: Waiting for application startup.

+-----+
| ADK Web Server started |
| For local testing, access at http://127.0.0.1:8000. |
+-----+

INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

Click this link to open ADK Web UI

點選網址後，您會看到 ADK 網頁使用者介面。請務必從左上角選取「代理程式」。



2. 測試搜尋功能

這項代理程式會智慧地轉送您的查詢。在對話視窗中輸入下列內容，即可查看不同的搜尋方式。

A. 圖表 RAG (語意搜尋)

根據語意和概念尋找項目，即使關鍵字不相符也沒關係。

測試查詢：(選擇下列任一項)

Who can help with injuries?

What abilities are related to survival?

報表重點：

- 推論過程應提及語意或 **RAG** 搜尋。
- 您應該會看到概念相關的結果 (例如，詢問「急救」時，會看到「手術」)。
- 結果會顯示  圖示。

B. 混合搜尋

結合關鍵字篩選條件與語意解讀，處理複雜查詢。

測試查詢：(選擇下列任一項)

Find someone who can fly a plane in the volcanic area

Who has healing abilities in the FOSSILIZED?

Who has healing abilities in the mountains?

報表重點：

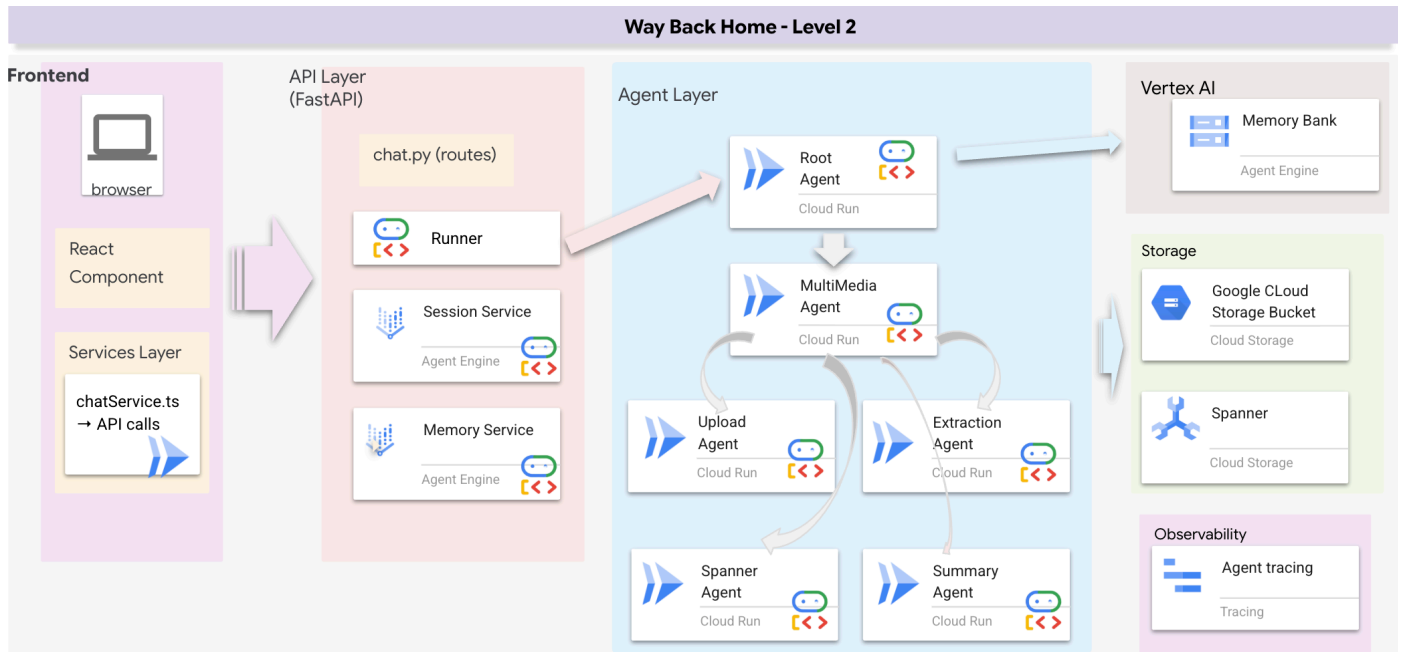
- 推論過程應提及混合式搜尋。
- 結果應符合「概念」和「地點/類別」這兩項條件。
- 兩種方法找到的結果都會顯示  圖示，且排名最高。

  測試完成後，請在指令列中按下 `Ctrl+C` 結束程序。

步驟 8 · 約 0 分鐘

8. 🚀 執行完整應用程式

全端架構總覽



新增 SessionService 和 Runner

👉 在終端機中執行下列指令，在 Cloud Shell 編輯器中開啟 `chat.py` 檔案 (請務必先按「Ctrl+C」結束上一個程序，再繼續操作)：

```
cloudshell edit ~/way-back-home/level_2/backend/api/routes/chat.py
```

👉 在 `chat.py` 檔案中，找出註解 `# TODO: REPLACE_INMEMORY_SERVICES`，將整行程式碼替換為下列程式碼：

```
session_service = InMemorySessionService()
memory_service = InMemoryMemoryService()
```

👉 在 `chat.py` 檔案中，找出註解 `# TODO: REPLACE_RUNNER`，將整行程式碼替換為下列程式碼：

```
runner = Runner(
    agent=root_agent,
    session_service=session_service,
    memory_service=memory_service,
    app_name="survivor-network"
)
```

1. 開始申請

如果先前的終端機仍在執行，請按下 `Ctrl+C` 結束。

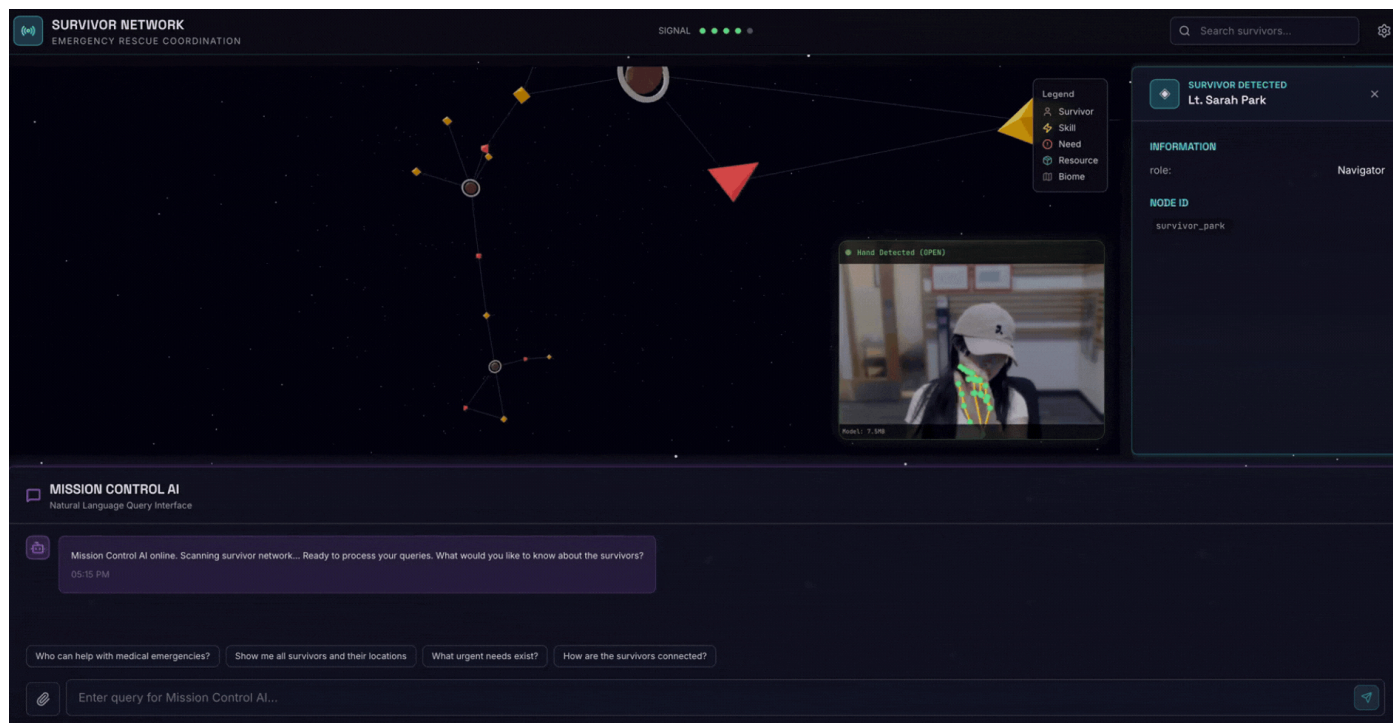
👉 啟動應用程式：

```
cd ~/way-back-home/level_2/  
./start_app.sh
```

後端成功啟動後，您會看到如下所示的 `Local: http://localhost:5173/"`：

```
npm warn unknown project config always-true: this will  
  
> frontend@0.0.0 dev  
> vite  
  
VITE v7.3.1 ready in 332 ms  
  
→ Local: http://localhost:5173/  
→ Network: http://10.88.0.3:5173/  
→ press h + enter to show help
```

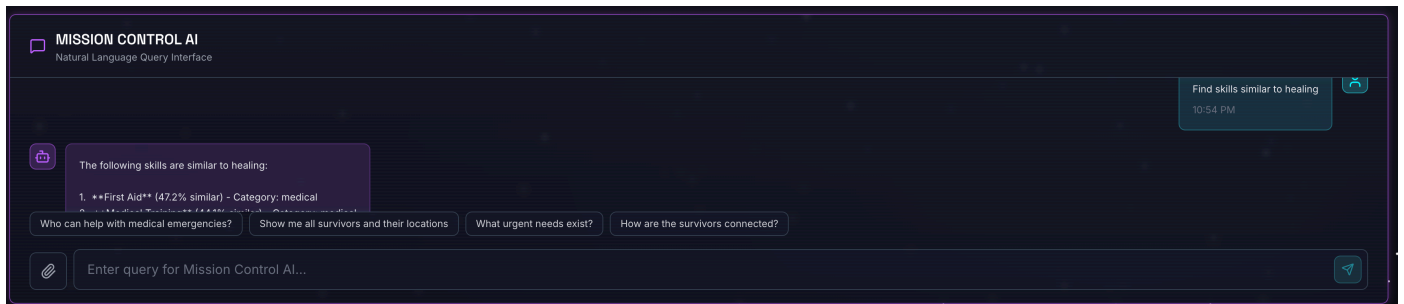
👉 按一下終端機中的「Local: http://localhost:5173/」。



2. 測試語意搜尋

查詢：

```
Find skills similar to healing
```



後續情形：

- 代理程式辨識相似度要求
- 為「療癒」生成嵌入
- 使用餘弦距離找出語意相似的技能
- 傳回：急救 (即使名稱與「治療」不符)

3. 測試混合型搜尋

查詢：

```
Find medical skills in the mountains
```

後續情形：

1. 關鍵字元件：篩選 `category='medical'`
2. 語意元件：嵌入「醫療」並依相似度排序
3. 合併：合併結果，並優先顯示兩種方法都找到的結果

查詢(選用)：

```
Who is good at survival and in the forest?
```

後續情形：

- 關鍵字搜尋結果：`biome='forest'`
- 語意搜尋結果：與「生存」類似的技能
- 混合式方法結合兩者，可獲得最佳成效

👉 測試完成後，請在終端機中按 `Ctrl+C` 結束測試。

4. (!ONLY FOR WORKSHOP ATTENDEE) Update your location

👉 執行完成指令碼：

```
cd ~/way-back-home/level_2
./set_level_2.sh
```

如果尚未參加研討會並完成第 0 級，系統會顯示錯誤訊息。

現在開啟 `waybackhome.dev`，您會發現位置資訊已更新。恭喜完成第 2 級！



9. ☕ [Optional] Multimodal Pipeline (Read Only) — Tooling Layer

為什麼需要多模態管道？

生存網路不只是文字，現場的倖存者可直接透過即時通訊傳送非結構化資料：

- 📷 圖片：資源、危害或設備的相片
- 🎥 影片：狀態報告或 SOS 廣播
- 📄 文字：現場筆記或記錄

挑戰：如要讓這項資料可供搜尋（「水在哪裡？」），我們需要從這些檔案擷取結構化實體（人員、資源、位置），並儲存至圖形。

現實生活情境：

使用者在對話中上傳醫療包的相片。

系統必須：

1. 將檔案上傳至 GCS
2. 使用 Gemini 「查看」 醫療包
3. 將「Resource」節點儲存至 Spanner
4. 向使用者確認：「我已將醫療包儲存至資料庫。」

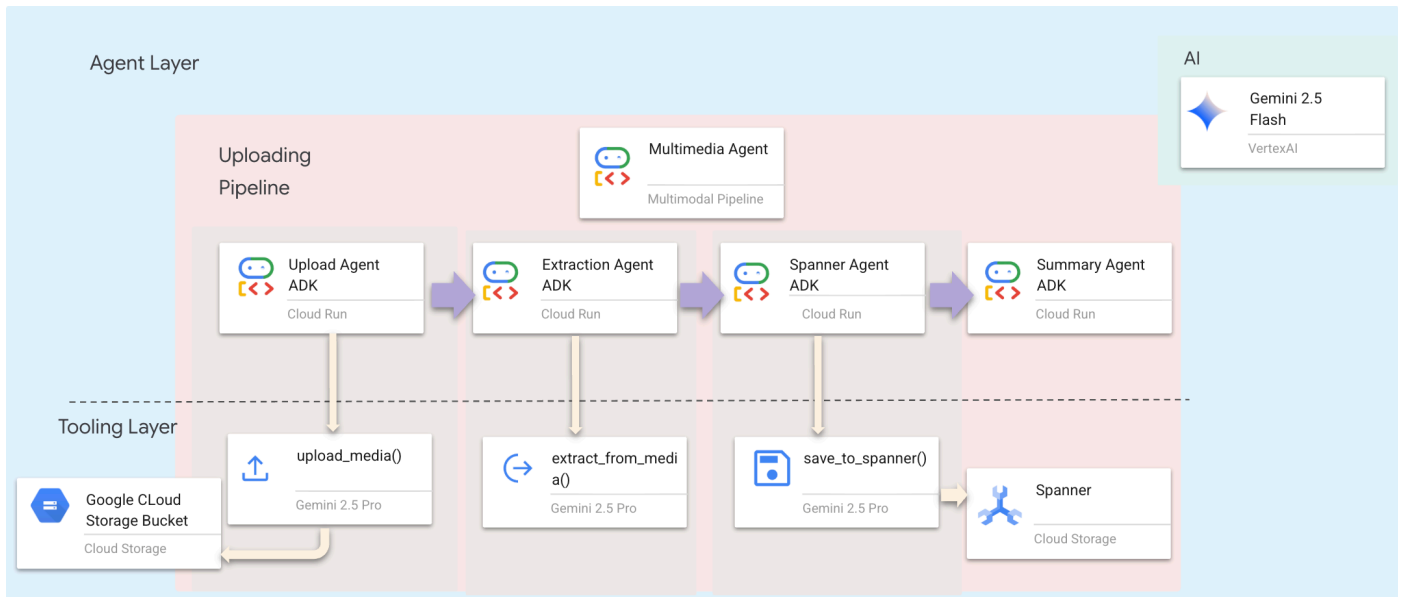
我們會處理哪些檔案？

與先前搜尋現有資料的步驟不同，這裡會處理**使用者上傳的檔案**。 `chat.py` 介面會動態處理檔案附件：

來源	內容	目標
使用者附件	圖片/影片/文字	要新增至圖表的資訊
對話內容	「這是用品的照片」	意圖和其他詳細資料

規劃的做法：循序代理管道

我們使用 **Sequential Agent** (`multimedia_agent.py`)，將專業代理串連在一起：



這是在 `backend/agent/multimedia_agent.py` 中定義為 `SequentialAgent`。

工具層提供代理可叫用的功能。工具會處理「如何」上傳檔案、擷取實體，以及儲存至資料庫。

1. 開啟工具檔案

👉 開啟 `level_2/backend/agent/tools/extraction_tools.py` 檔案，或在終端機中輸入下列指令。開啟新的終端機。在終端機中，於 Cloud Shell 編輯器開啟檔案：

```
cloudshell edit ~/way-back-home/level_2/backend/agent/tools/extraction_tools.py
```

2. 導入 `upload_media` 工具

這項工具會將本機檔案上傳至 Google Cloud Storage。

👉 在 `def upload_media(file_path: str, survivor_id: Optional[str] = None) -> Dict[str, Any]:` 中，下列程式碼說明如何將檔案上傳至 GCS 並偵測檔案類型：


```

"""
Upload media file to GCS and detect its type.

Args:
    file_path: Path to the local file
    survivor_id: Optional survivor ID to associate with upload

Returns:
    Dict with gcs_uri, media_type, and status
"""
try:
    if not file_path:
        return {"status": "error", "error": "No file path provided"}

    # Strip quotes if present
    file_path = file_path.strip().strip('"').strip('\'')

    if not os.path.exists(file_path):
        return {"status": "error", "error": f"File not found: {file_path}"}

    gcs_uri, media_type, signed_url = gcs_service.upload_file(file_path, survivor_id)

    return {
        "status": "success",
        "gcs_uri": gcs_uri,
        "signed_url": signed_url,
        "media_type": media_type.value,
        "file_name": os.path.basename(file_path),
        "survivor_id": survivor_id
    }
except Exception as e:
    logger.error(f"Upload failed: {e}")
    return {"status": "error", "error": str(e)}

```

3. 導入 `extract_from_media` 工具

這項工具是**路由器**，會檢查 `media_type` 並派送至正確的擷取器 (文字、圖片或影片)。

👉 在 `async def extract_from_media(gcs_uri: str, media_type: str, signed_url: Optional[str] = None) -> Dict[str, Any]:` 中，下列程式碼說明如何從上傳的媒體擷取實體和關係。

```

"""
Extract entities and relationships from uploaded media.

Args:
    gcs_uri: GCS URI of the uploaded file
    media_type: Type of media (text/image/video)
    signed_url: Optional signed URL for public/temporary access

Returns:
    Dict with extraction results
"""
try:
    if not gcs_uri:
        return {"status": "error", "error": "No GCS URI provided"}

    # Select appropriate extractor
    if media_type == MediaType.TEXT.value or media_type == "text":
        result = await text_extractor.extract(gcs_uri)
    elif media_type == MediaType.IMAGE.value or media_type == "image":
        result = await image_extractor.extract(gcs_uri)
    elif media_type == MediaType.VIDEO.value or media_type == "video":
        result = await video_extractor.extract(gcs_uri)
    else:
        return {"status": "error", "error": f"Unsupported media type: {media_type}"}

    # Inject signed URL into broadcast info if present
    if signed_url:
        if not result.broadcast_info:
            result.broadcast_info = {}
        result.broadcast_info['thumbnail_url'] = signed_url

    return {
        "status": "success",
        "extraction_result": result.to_dict(), # Return valid JSON dict instead of object
        "summary": result.summary,
        "entities_count": len(result.entities),
        "relationships_count": len(result.relationships),
        "entities": [e.to_dict() for e in result.entities],
        "relationships": [r.to_dict() for r in result.relationships]
    }
except Exception as e:
    logger.error(f"Extraction failed: {e}")
    return {"status": "error", "error": str(e)}

```

重要導入詳細資料：

- **多模態輸入**：我們會將文字提示 (`_get_extraction_prompt()`) 和圖片物件傳遞至 `generate_content`。
- **結構化輸出內容**：`response_mime_type="application/json"` 確保 LLM 傳回有效的 JSON，這對管道至關重要。
- **視覺實體連結**：提示包含已知實體，因此 Gemini 可以辨識特定字元。

4. 導入 `save_to_spanner` 工具

這項工具會將擷取的實體和關係儲存至 Spanner Graph 資料庫。

👉 在 `def save_to_spanner(extraction_result: Any, survivor_id: Optional[str] = None) -> Dict[str, Any]` 中，下列程式碼說明如何將擷取的實體和關係儲存至 Spanner Graph 資料庫。

```
"""
Save extracted entities and relationships to Spanner Graph DB.

Args:
    extraction_result: ExtractionResult object (or dict from previous step if passed as dict)
    survivor_id: Optional survivor ID to associate with the broadcast

Returns:
    Dict with save statistics
"""
try:
    # Handle if extraction_result is passed as the wrapper dict from extract_from_media
    result_obj = extraction_result
    if isinstance(extraction_result, dict) and 'extraction_result' in extraction_result:
        result_obj = extraction_result['extraction_result']

    # If result_obj is a dict (from to_dict()), reconstruct it
    if isinstance(result_obj, dict):
        from extractors.base_extractor import ExtractionResult
        result_obj = ExtractionResult.from_dict(result_obj)

    if not result_obj:
        return {"status": "error", "error": "No extraction result provided"}

    stats = spanner_service.save_extraction_result(result_obj, survivor_id)

    return {
        "status": "success",
        "entities_created": stats['entities_created'],
        "entities_existing": stats['entities_found_existing'],
        "relationships_created": stats['relationships_created'],
        "broadcast_id": stats['broadcast_id'],
        "errors": stats['errors'] if stats['errors'] else None
    }
except Exception as e:
    logger.error(f"Spanner save failed: {e}")
    return {"status": "error", "error": str(e)}
```

我們提供高階工具給代理程式，確保資料完整性，同時運用代理程式的推理能力。

5. 更新 GCS 服務

`GCSService` 會處理實際的檔案上傳至 Google Cloud Storage 作業。

👉 開啟 `level_2/backend/services/gcs_service.py` 檔案，或在終端機中輸入指令，在 Cloud Shell 編輯器中開啟檔案：

```
cloudshell edit ~/way-back-home/level_2/backend/services/gcs_service.py
```

👉 在 `def upload_file(self, file_path: str, survivor_id: Optional[str] = None) -> Tuple[str, MediaType, str]:` 中，下列程式碼說明如何將擷取的實體和關係儲存至 Spanner Graph 資料庫。

```
blob = self.bucket.blob(blob_name)
blob.upload_from_filename(file_path)
```

將這項作業抽象化為服務後，代理程式就不需要瞭解 GCS bucket、Blob 名稱或經簽署的網址產生方式。系統只會要求「上傳」。

6. 為什麼要使用代理工作流程 > 傳統方法？

傳統上，有兩種方法可以建構這個管道。這兩種方法都有重大缺點，而代理程式工作流程可解決這些問題。

✗ 方法 1：批次資料管道 (簡單但脆弱)

以下是依序處理檔案的典型批次指令碼：

```
# Traditional batch pipeline
for file in ["broadcast_1.txt", "broadcast_2.png", "broadcast_3.mp4"]:
    gcs_uri = upload_to_gcs(file)          # Fails if file corrupted → CRASH
    entities = extract_entities(gcs_uri)    # No error handling
    save_to_db(entities)                   # Saves even if extraction empty → WASTE
    # How do you tell the user progress? Print to console?
```

問題：

- ✗ 無法適應：如果一個檔案失敗，整個指令碼就會當機
- ✗ 沒有脈絡：無法根據使用者意圖調整行為 (「這很緊急！」)
- ✗ 無聲失敗：記錄中隱藏的錯誤，使用者不知道發生了什麼事
- ✗ 沒有使用者意見回饋：指令碼在背景執行，使用者只能盲等

✗ 方法 2：事件導向架構 (雲端原生，但較為複雜)

傳統雲端設定：

1. 使用者上傳至 GCS
2. GCS 事件會觸發 Cloud 函式
3. 函式呼叫 Vision API
4. 函式會寫入資料庫
5. ...How do we tell the user it finished? (需要 WebSockets/輪詢)

問題：

- ✗ 高複雜度：管理 5 項以上的服務 (GCS、Eventarc、Cloud Functions、Pub/Sub...)
- ✗ 狀態已分離：難以在事件處理常式之間傳遞資料
- ✗ 偵錯惡夢：記錄分散在多項服務中
- ✗ 與使用者中斷連線：活動在背景執行，沒有對話內容

✅ 我們的方法：代理循序管道

`multimedia_agent.py` 使用 **SequentialAgent** 智慧地協調管道：

```
# Agentic pipeline (simplified conceptual view)
Agent: "I'll upload your file..."
Tool: upload_media → Success ✅
Agent: "Great! Now extracting entities..."
Tool: extract_from_media → Found 3 survivors, 2 resources ✅
Agent: "Perfect! Saving to database..."
Tool: save_to_spanner → Saved as broadcast #456 ✅
Agent: "Done! I found 3 survivors and 2 resources in your image. Saved to the graph."
```

代理優勢：

功能	批次管道	事件導向	代理工作流程
複雜度	低 (1 個指令碼)	高 (5 項以上的服務)	低 (1 個 Python 檔案： <code>multimedia_agent.py</code>)
狀態管理	全域變數	硬 (已解除連結)	整合 (專員狀態)
錯誤處理	當機	無聲記錄	互動式 (「我無法讀取該檔案」)
使用者意見回饋	無框畫	需要輪詢	立即 (對話的一部分)
適應性	修正邏輯	嚴格函式	智慧 (LLM 決定下一步)
情境感知	無	無	完整 (瞭解使用者意圖)

實際範例：

批次指令碼：

```
Processing file 1... Done.
Processing file 2... ERROR: Corrupted image
[CRASH - User has to restart entire batch]
```

代理工作流程：

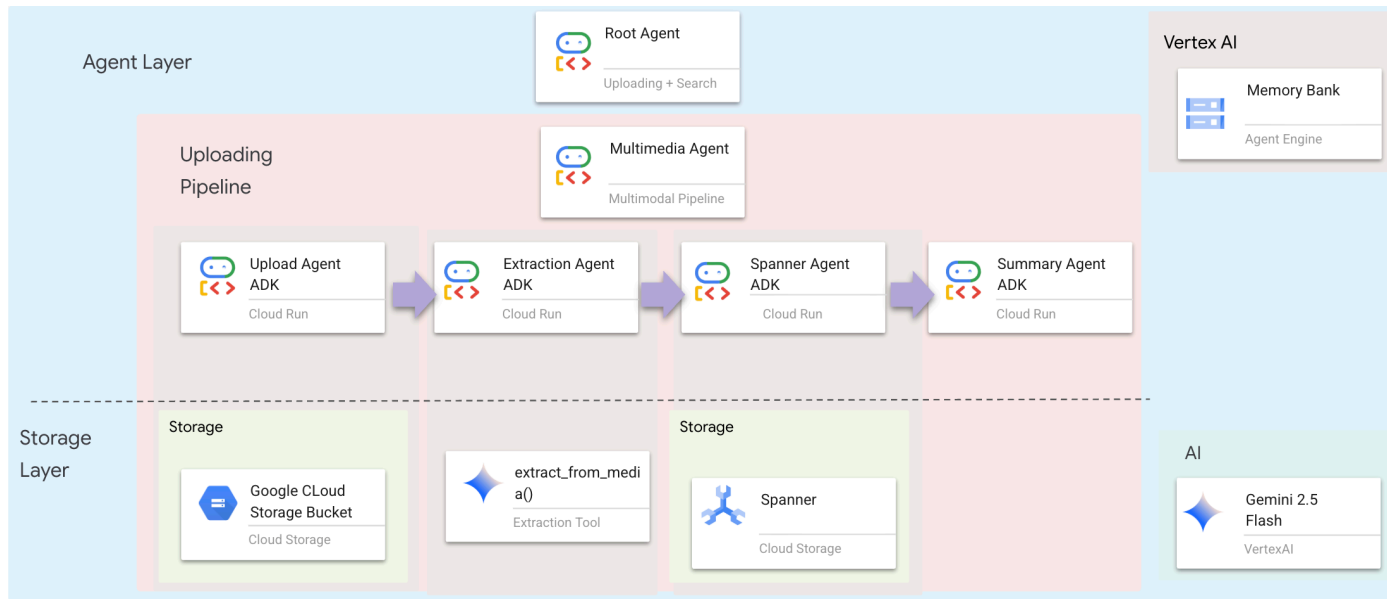
```
User: "Here are 3 images of our supply cache"
Agent: "Processing image 1... Found medical supplies ✅"
Agent: "Processing image 2... This file seems corrupted. Skipping. ⚠️"
Agent: "Processing image 3... Found food supplies ✅"
Agent: "Summary: I successfully processed 2 of 3 images. Would you like to re-upload image 2?"
```

重要性：使用 `multimedia_agent.py` (具有 4 個子代理程式的 SequentialAgent：上傳 → 擷取 → 儲存 → 摘要)，我們以智慧型對話應用程式邏輯取代複雜的基礎架構和脆弱的指令碼。

步驟 10 · 約 0 分鐘

10. ☕ [Optional] Multimodal Pipeline (Read Only) — Agent Layer

代理程式層定義了智慧，也就是使用工具完成工作的代理程式。每個代理程式都有特定角色，並將脈絡傳遞給下一個代理程式。下圖為多代理系統的架構圖。



1. 開啟代理程式檔案

👉 開啟 `level_2/backend/agent/multimedia_agent.py` 檔案，或在終端機中輸入下列指令。開啟新的終端機。在終端機中，於 Cloud Shell 編輯器開啟檔案：

```
cloudshell edit ~/way-back-home/level_2/backend/agent/multimedia_agent.py
```

2. 定義上傳代理程式

這個代理會從使用者訊息中擷取檔案路徑，然後上傳至 GCS。

👉 在 `multimedia_agent.py` 檔案中，使用下列程式碼建立 `upload_agent`，上傳至 GCS：

```
upload_agent = LlmAgent(  
    name="UploadAgent",  
    model="gemini-2.5-flash",  
    instruction="""Extract the file path from the user's message and upload it.  
  
Use `upload_media(file_path, survivor_id)` to upload the file.  
The survivor_id is optional - include it if the user mentions a specific survivor (e.g.,  
"survivor Sarah" -> "Sarah").  
If the user provides a path like "/path/to/file", use that.  
  
Return the upload result with gcs_uri and media_type."""  
    tools=[upload_media],  
    output_key="upload_result"  
)
```


3. 定義擷取代理程式

這個代理程式會「看到」上傳的媒體，並使用 Gemini Vision 擷取結構化資料。

👉 在 `multimedia_agent.py` 檔案中，使用下列程式碼建立 `extraction_agent`，從上傳的媒體中擷取資訊：

```
extraction_agent = LlmAgent(
    name="ExtractionAgent",
    model="gemini-2.5-flash",
    instruction="""Extract information from the uploaded media.

Previous step result: {upload_result}

Use `extract_from_media(gcs_uri, media_type, signed_url)` with the values from the upload
result.
The gcs_uri is in upload_result['gcs_uri'], media_type in upload_result['media_type'], and
signed_url in upload_result['signed_url'].

Return the extraction results including entities and relationships found.""",
    tools=[extract_from_media],
    output_key="extraction_result"
)
```

請注意 `instruction` 如何參照 `{upload_result}`，這是在 ADK 中代理之間傳遞狀態的方式。

4. 定義 Spanner 代理程式

這個代理程式會將擷取的實體和關係儲存至圖形資料庫。

👉 在 `multimedia_agent.py` 檔案中，使用下列程式碼建立 `spanner_agent`，將擷取的資訊儲存至資料庫：

```
spanner_agent = LlmAgent(
    name="SpannerAgent",
    model="gemini-2.5-flash",
    instruction="""Save the extracted information to the database.

Upload result: {upload_result}
Extraction result: {extraction_result}

Use `save_to_spanner(extraction_result, survivor_id)` to save to Spanner.
Pass the WHOLE `extraction_result` object/dict from the previous step.
Include survivor_id if it was provided in the upload step.

Return the save statistics.""",
    tools=[save_to_spanner],
    output_key="spanner_result"
)
```

這個代理程式會從先前步驟 (`upload_result` 和 `extraction_result`) 接收脈絡資訊。

5. 定義摘要代理程式

這個代理程式會將先前所有步驟的結果統整成易於理解的回覆。

👉 在 `multimedia_agent.py` 檔案中，使用下列程式碼定義 `summary_agent` 的提示，以摘要說明結果：

```
USE_MEMORY_BANK = os.getenv("USE_MEMORY_BANK", "false").lower() == "true"
save_msg = "6. Mention that the data is also being synced to the memory bank." if
USE_MEMORY_BANK else ""

summary_instruction = f"""Provide a user-friendly summary of the media processing.

Upload: {{upload_result}}
Extraction: {{extraction_result}}
Database: {{spanner_result}}

Summarize:
1. What file was processed (name and type)
2. Key information extracted (survivors, skills, needs, resources found) - list names and counts
3. Relationships identified
4. What was saved to the database (broadcast ID, number of entities)
5. Any issues encountered
{save_msg}

Be concise but informative."""
```

這個代理程式不需要工具，只要讀取共用的內容，就能為使用者產生簡潔的摘要。

架構摘要

圖層	檔案	責任
工具	<code>extraction_tools.py</code> + <code>gcs_service.py</code>	如何：上傳、擷取、儲存
Agent	<code>multimedia_agent.py</code>	內容：自動調度管理管道

11. 🚀 多模態資料管道 - 自動化調度管理

新系統的核心是 `backend/agent/multimedia_agent.py`，定義於 `MultimediaExtractionPipeline`。這項功能採用 ADK (Agent Development Kit) 的序列代理模式。

1. 為什麼要使用連續式廣告？

處理上傳內容時，會依序執行下列步驟：

1. 您必須先上傳檔案，才能擷取資料。
2. 您必須先擷取資料，才能儲存資料。
3. 您必須先儲存結果，才能產生摘要。

這時很適合使用 `SequentialAgent`，並將一個代理程式的輸出內容做為下一個代理程式的脈絡/輸入內容。

2. 代理程式定義

讓我們看看 `multimedia_agent.py` 底部的管道組裝方式：👉🖥️ 在終端機中執行下列指令，在 Cloud Shell 編輯器中開啟檔案：

```
cloudshell edit ~/way-back-home/level_2/backend/agent/multimedia_agent.py
```

並接收兩個先前步驟的輸入內容。找出註解 `# TODO: REPLACE_ORCHESTRATION`。將整行程式碼替換為下列程式碼：

```
sub_agents=[upload_agent, extraction_agent, spanner_agent, summary_agent]
```

3. 與根代理程式連線

👉🖥️ 在終端機中執行下列指令，在 Cloud Shell 編輯器中開啟檔案：

```
cloudshell edit ~/way-back-home/level_2/backend/agent/agent.py
```

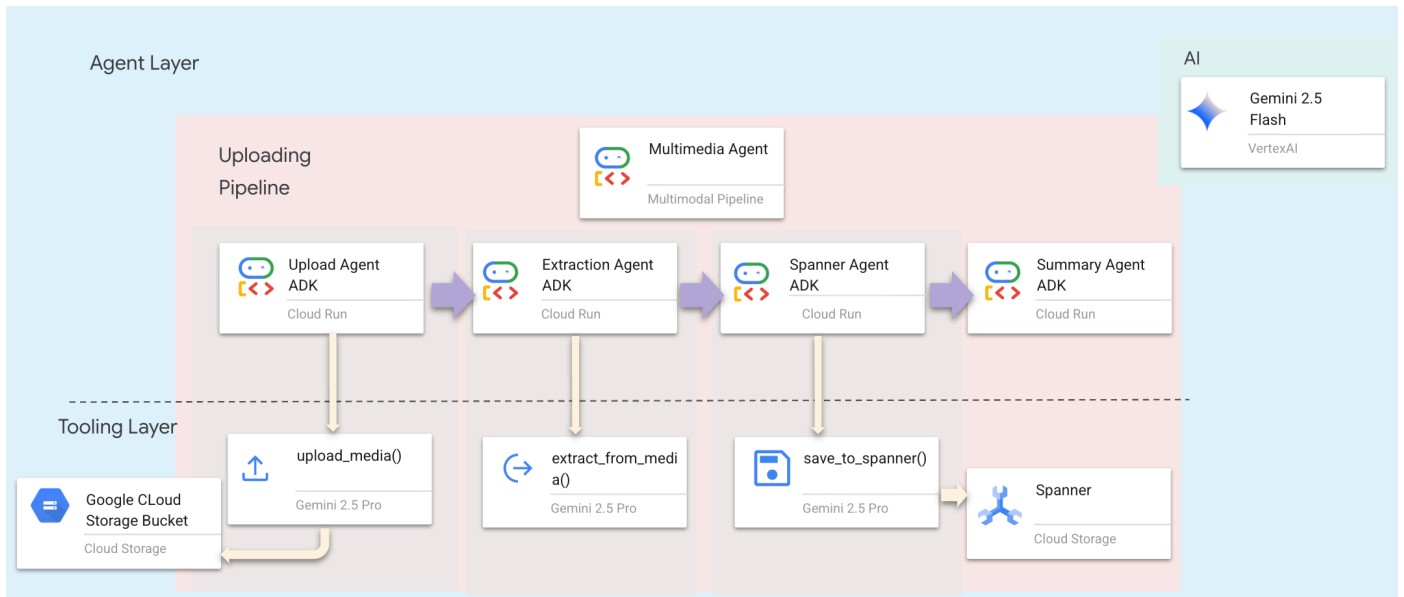
找出註解 `# TODO: REPLACE_ADD_SUBAGENT`。將整行程式碼替換為下列程式碼：

```
sub_agents=[multimedia_agent],
```

這個單一物件會將四個「專家」有效綁定為一個可呼叫的實體。

4. 代理之間的資料流

每個代理程式都會將輸出內容儲存在共用的環境中，後續代理程式可以存取：



5. 開啟應用程式 (如果應用程式仍在執行中，請略過此步驟)

如果先前的應用程式程序仍處於開啟狀態，可以略過這個步驟。

👉 啟動應用程式：

```
cd ~/way-back-home/level_2/  
./start_app.sh
```

👉 按一下終端機中的「Local: http://localhost:5173/」。

6. 測試圖片上傳

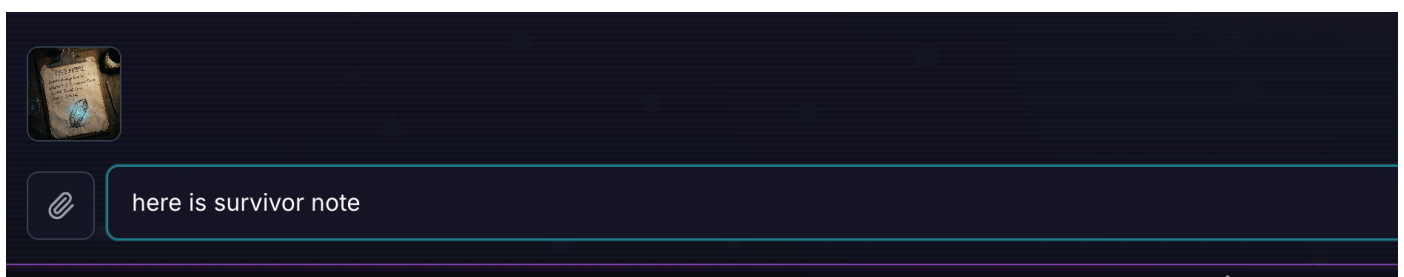
👉 在對話介面中選擇任一相片，然後上傳至使用者介面：

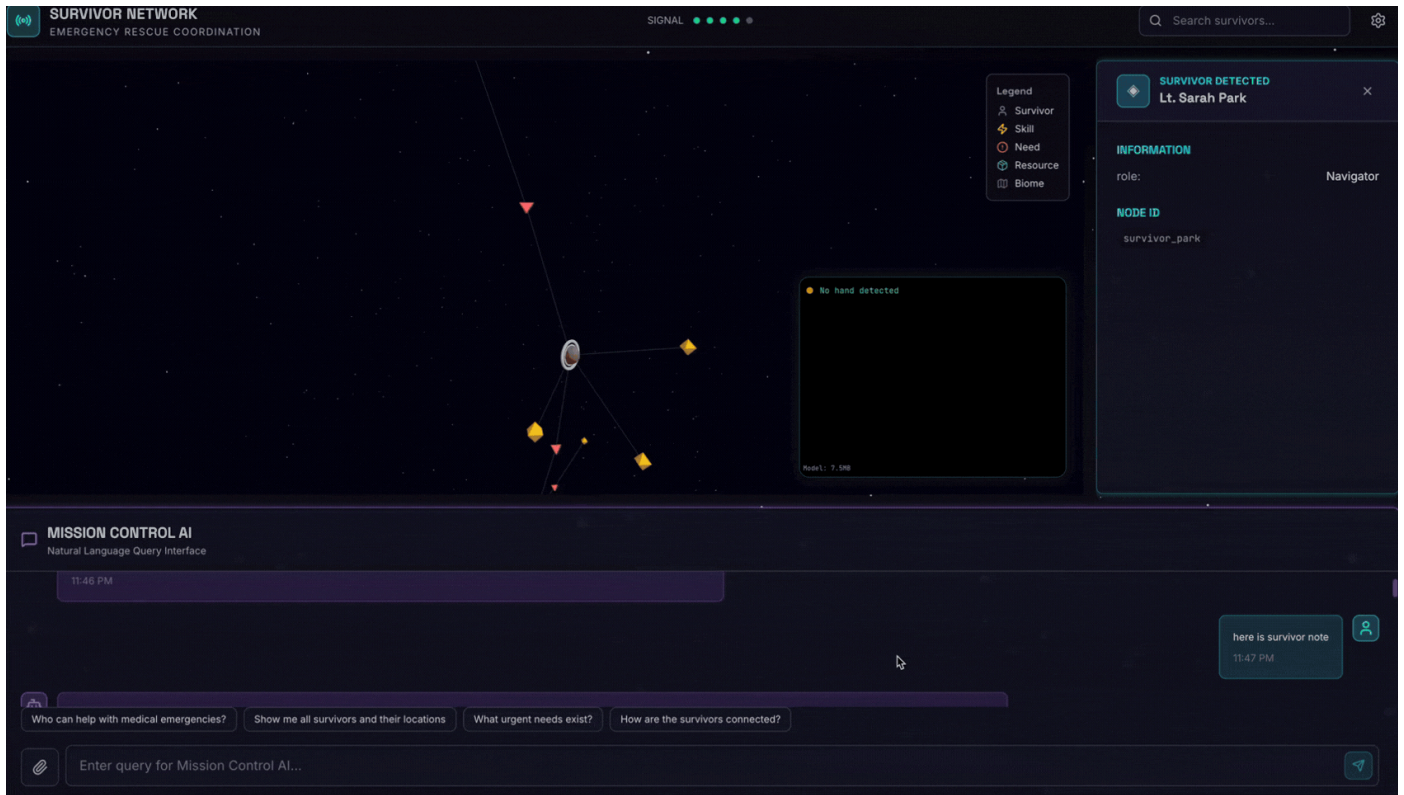
- [test_photo1](#),
- [test_photo2](#),
- [test_photo3](#)、
- [test_photo4](#)

在對話介面中，向代理程式說明具體情況：

Here is the survivor note

然後在這裡附上圖片。





👉 測試完成後，在終端機中按下「Ctrl+C」結束程序。

幕後到底發生了什麼事：

1. UploadAgent：

- 將檔案上傳至 Google Cloud Storage
- 偵測媒體類型 (圖片/文字/影片)
- 產生用於存取的經簽署的網址

2. ExtractionAgent：

- 從 GCS 下載圖片
- 將圖片連同擷取提示傳送給 Gemini Vision
- AI 分析圖片內容：
 - 生還者 (姓名、狀況、角色)
 - 資源 (醫療用品、工具)
 - 地點 (生態域、座標)
 - 關係 (誰擁有什麼、誰在哪裡)

3. SpannerAgent：

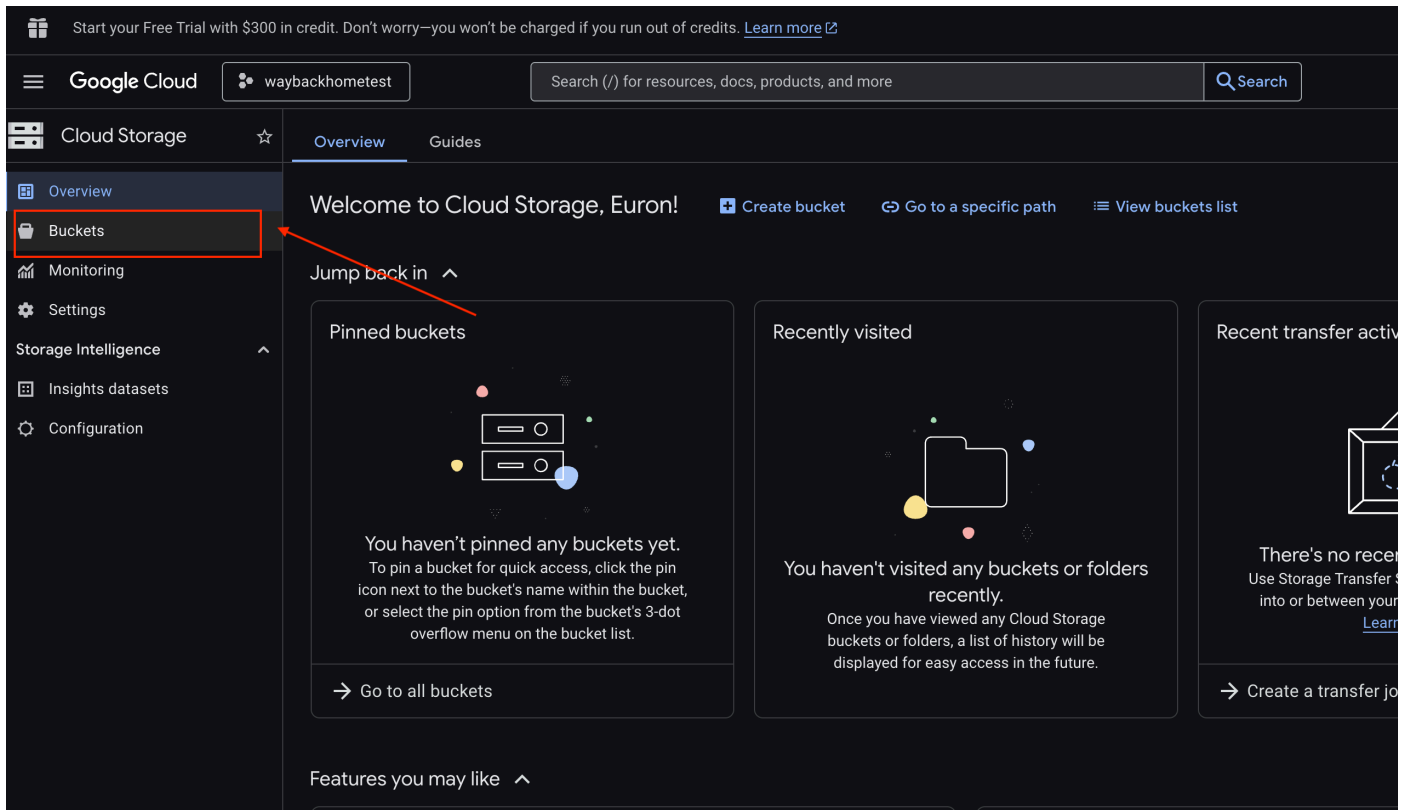
- 在 Spanner 中建立新的實體節點
- 建立關係邊緣
- 建立含有縮圖的廣播記錄

4. SummaryAgent：

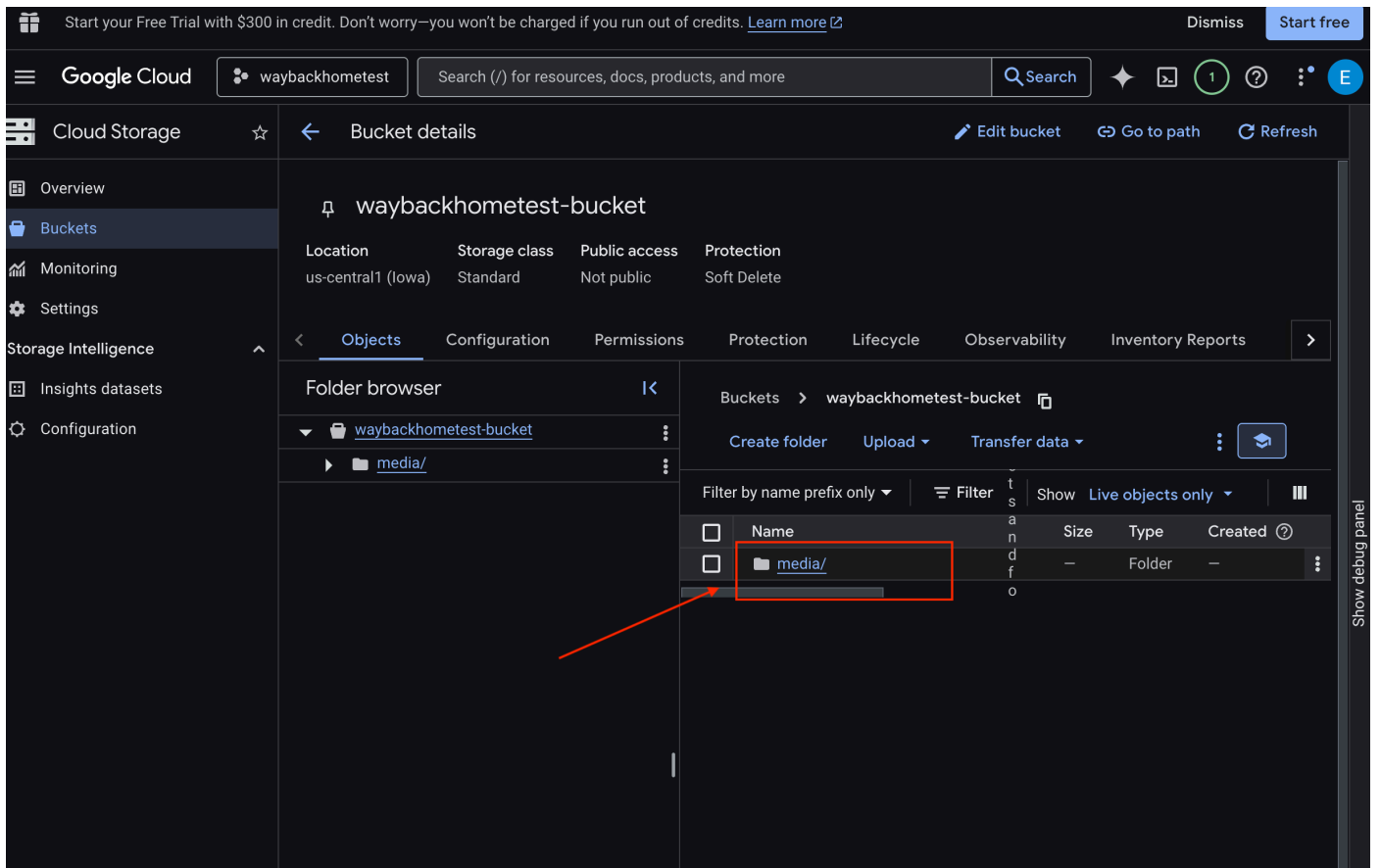
- 生成簡單易懂的摘要
- 傳回結構化報表

6. 在 GCS 值區中驗證多模態上傳作業

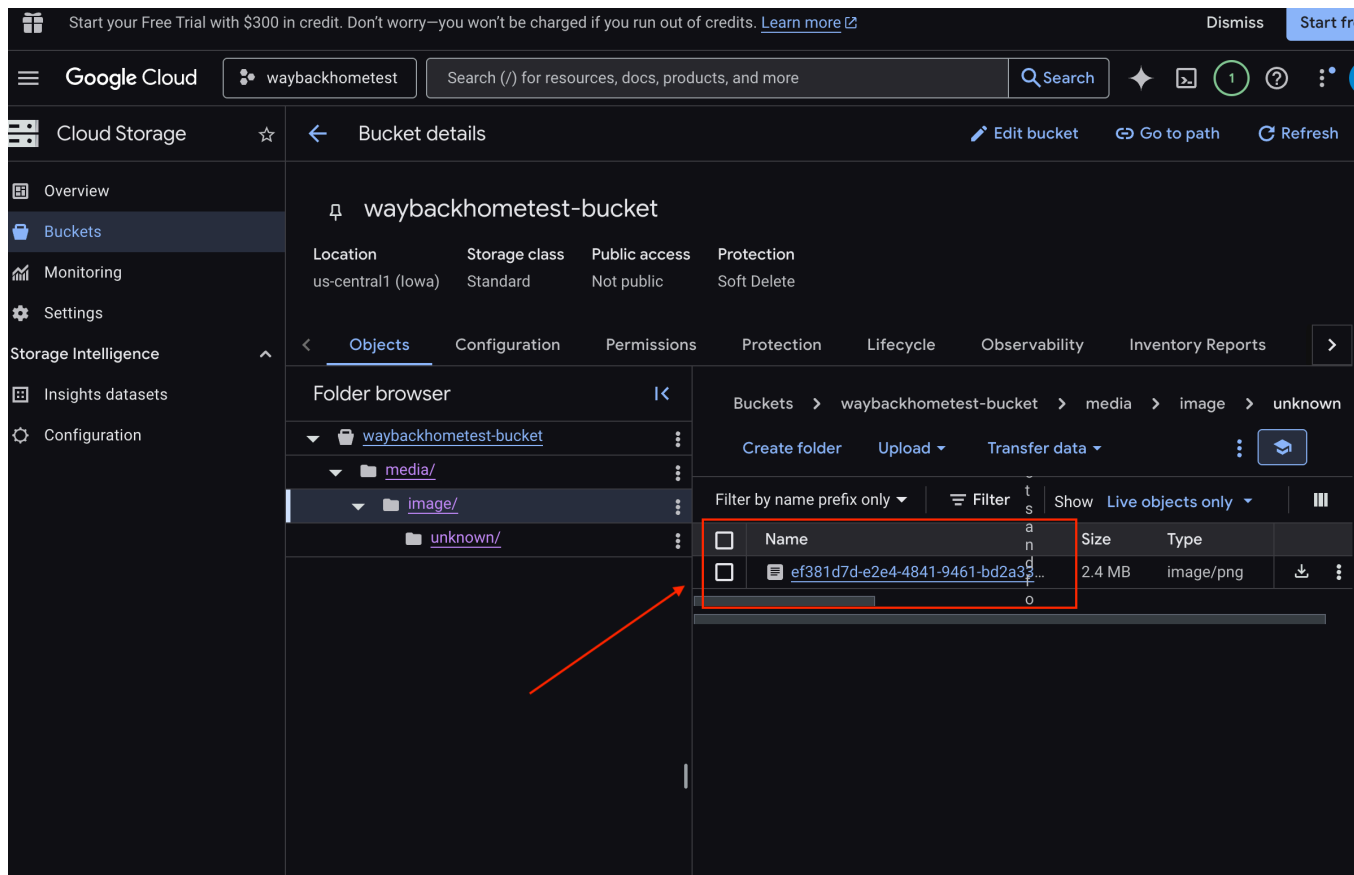
- 開啟 [Google Cloud 控制台儲存空間](#)。
- 選取 Cloud Storage 中的「bucket」



- 選取 bucket，然後按一下 `media`。



- 您上傳的圖片會顯示在這裡。



7. 在 Spanner 中驗證多模態上傳 (選用)

以下是 `test_photo1` 的 UI 輸出範例。

上傳 `test_photo1` 並附上附註 `Here is the survivor note` 後，你可以查看訊息

```
**Summary:** A field report written on a clipboard details the discovery of an 'Energy Crystal' by David Chen in a 'Bioluminescent Forest', with the status marked as 'Critical'. The scene also includes a lit lantern, a pen, and a compass.
```

```
**Entities Found:**
```

```
* **David Chen** (Survivor): The agent who filed this field report. His known role is Engineer.
```

```
* **Engineer** (Skill): David Chen's known role or skill.
```

```
* **Energy Crystal** (Resource): A newly discovered resource, depicted as a glowing blue crystal in a sketch on the report. Its purpose is implied to be energy-related.
```

```
* **Bioluminescent Forest** (Biome): The location where the energy crystal was found. This matches the known biome description of a dark forest with glowing purple/neon plants and mushrooms.
```

```
* **Critical Situation** (Need): The overall status of the situation is critical, indicating an urgent need for attention, response, or resource allocation.
```

```
**Relationships Found:**
```

```
* **David Chen** (Survivor) found **Energy Crystal** (Resource).
```

```
* **David Chen** (Survivor) is in **Bioluminescent Forest** (Biome).
```

```
* **David Chen** (Survivor) has the skill **Engineer**.I have successfully saved the extracted information to the Spanner database.
```

```
Here are the save statistics:
```

```
* **Broadcast ID:** `5892fb58-a120-46ca-80c2-0e04da7d6ea7`
```

```
* **Entities Created:** 4
```

```
* **Existing Entities:** 1
```

```
* **Relationships Created:** 3
```

```
* **Status:** successHere's a summary of the media processing:
```

在這種情況下，我們需要驗證 Spanner 是否已成功更新下列資訊：

```
* **David Chen** (Survivor) found **Energy Crystal** (Resource).  
* **David Chen** (Survivor) is in **Bioluminescent Forest** (Biome).  
* **David Chen** (Survivor) has the skill **Engineer**.I have
```

- 開啟 [Google Cloud 控制台 Spanner](#)。
- 選取執行個體：`Survivor Network`
- 選取資料庫：`graph-db`
- 按一下左側邊欄中的「Spanner Studio」。

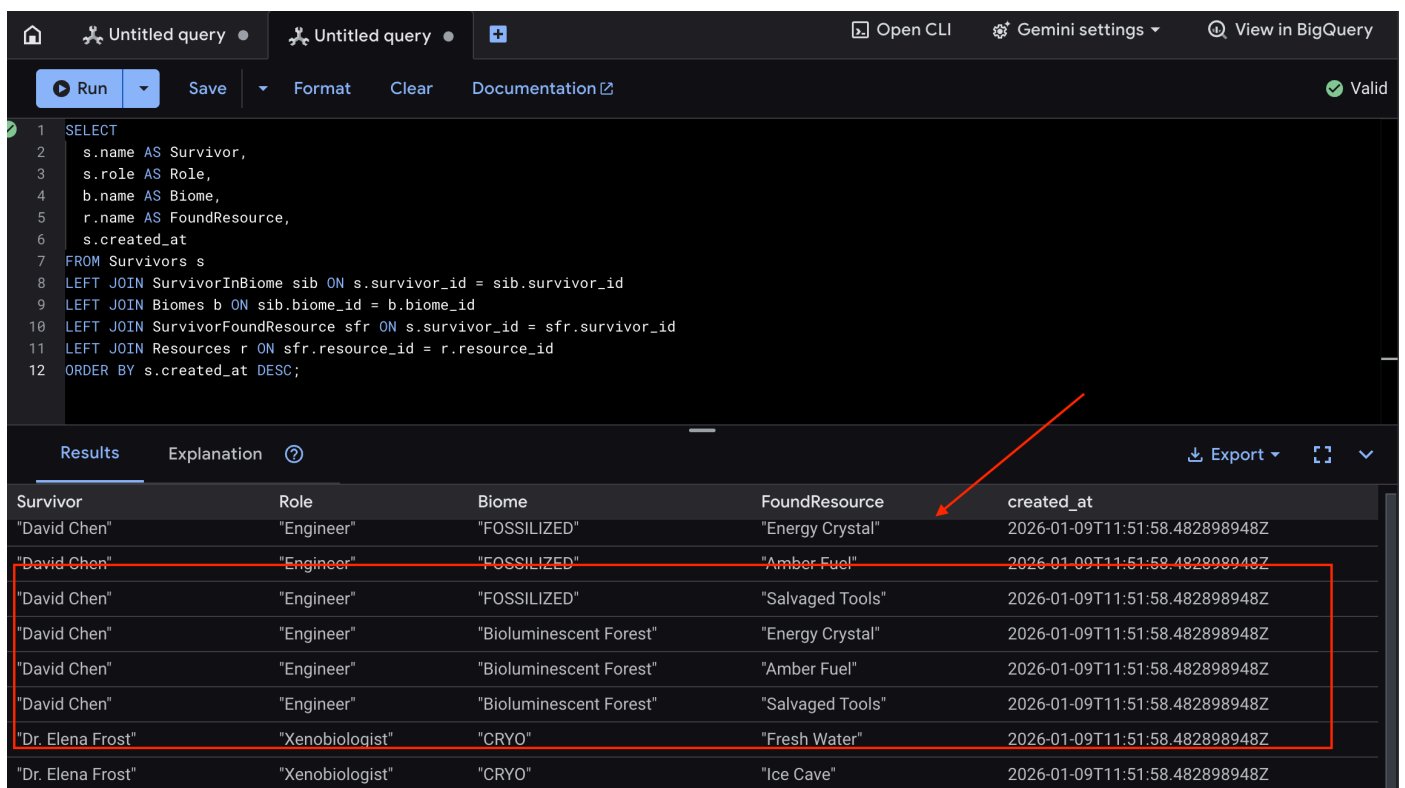
👉 在 Spanner Studio 中查詢新資料：

```

SELECT
  s.name AS Survivor,
  s.role AS Role,
  b.name AS Biome,
  r.name AS FoundResource,
  s.created_at
FROM Survivors s
LEFT JOIN SurvivorInBiome sib ON s.survivor_id = sib.survivor_id
LEFT JOIN Biomes b ON sib.biome_id = b.biome_id
LEFT JOIN SurvivorFoundResource sfr ON s.survivor_id = sfr.survivor_id
LEFT JOIN Resources r ON sfr.resource_id = r.resource_id
ORDER BY s.created_at DESC;

```

我們可以查看以下結果來驗證：



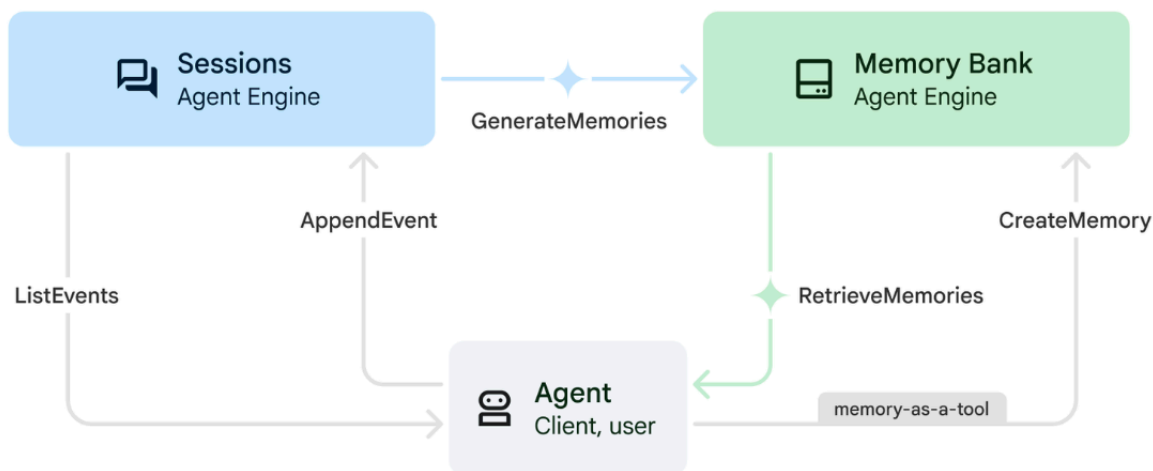
The screenshot shows a BigQuery interface with a SQL query and its results. The query is a SELECT statement joining Survivors, SurvivorInBiome, Biomes, SurvivorFoundResource, and Resources tables. The results table shows 8 rows of data. A red box highlights the first 7 rows, and a red arrow points to the 'Energy Crystal' resource in the first row.

Survivor	Role	Biome	FoundResource	created_at
"David Chen"	"Engineer"	"FOSSILIZED"	"Energy Crystal"	2026-01-09T11:51:58.482898948Z
"David Chen"	"Engineer"	"FOSSILIZED"	"Amber Fuel"	2026-01-09T11:51:58.482898948Z
"David Chen"	"Engineer"	"FOSSILIZED"	"Salvaged Tools"	2026-01-09T11:51:58.482898948Z
"David Chen"	"Engineer"	"Bioluminescent Forest"	"Energy Crystal"	2026-01-09T11:51:58.482898948Z
"David Chen"	"Engineer"	"Bioluminescent Forest"	"Amber Fuel"	2026-01-09T11:51:58.482898948Z
"David Chen"	"Engineer"	"Bioluminescent Forest"	"Salvaged Tools"	2026-01-09T11:51:58.482898948Z
"Dr. Elena Frost"	"Xenobiologist"	"CRYO"	"Fresh Water"	2026-01-09T11:51:58.482898948Z
"Dr. Elena Frost"	"Xenobiologist"	"CRYO"	"Ice Cave"	2026-01-09T11:51:58.482898948Z

12. ☕ [Optional] Memory Bank with Agent Engine

1. 記憶體運作方式

系統採用雙記憶體方法，同時處理即時脈絡和長期學習。



2. 什麼是記憶主題？

記憶體主題會定義代理程式在對話中應記住的資訊類別。這些檔案櫃會存放不同類型的使用者偏好設定。

為什麼不只用一個大型記憶體？

- ✗ 一般記憶內容：「使用者提到搜尋... 需要... 和生物群系...」
- ✓ 主題分類：清楚區分搜尋行為與緊急監控

我們的 2 個主題：

- search_preferences**：使用者偏好的搜尋方式
 - 他們偏好關鍵字搜尋還是語意搜尋？
 - 他們經常搜尋哪些技能/生物群系？
 - 記憶內容範例：「使用者偏好以語意搜尋方式使用醫療技能」
- urgent_needs_context**：他們追蹤的危機
 - 他們監控哪些資源？
 - 他們關心哪些倖存者？
 - 記憶內容範例：「使用者正在追蹤北營的藥物短缺情況」

為什麼是 2 個主題？

- **精確度**：LLM 能將事實歸類至正確類別
- **擷取**：使用者詢問「尋找具備醫療技能的倖存者」時，代理程式會從 `search_preferences` 擷取資料，而不是不相關的緊急需求資料
- **擴充性**：之後可以新增更多主題 (例如 `favorite_survivors`、`alliance_tracking`)

運作方式：與服務專員對話時，Memory Bank 會自動擷取符合這些主題描述的事實並儲存。下次開始新工作階段時，服務專員會擷取相關記憶內容，以便提供個人化回覆。

3. 設定回憶主題

自訂記憶體主題會定義代理程式應記住的内容。部署 Agent Engine 時會設定這些項目。

👉 在終端機中執行下列指令，在 Cloud Shell 編輯器中開啟檔案：

```
cloudshell edit ~/way-back-home/level_2/backend/deploy_agent.py
```

系統會在編輯器中開啟 `~/way-back-home/level_2/backend/deploy_agent.py`。

我們定義結構 `MemoryTopic` 物件，引導 LLM 擷取及儲存哪些資訊。

👉 在 `deploy_agent.py` 檔案中，將 `# TODO: SET_UP_TOPIC` 替換為下列內容：

```
# backend/deploy_agent.py

custom_topics = [
    # Topic 1: Survivor Search Preferences
    MemoryTopic(
        custom_memory_topic=CustomMemoryTopic(
            label="search_preferences",
            description="""Extract the user's preferences for how they search for
survivors. Include:
- Preferred search methods (keyword, semantic, direct lookup)
- Common filters used (biome, role, status)
- Specific skills they value or frequently look for
- Geographic areas of interest (e.g., "forest biome", "mountain outpost")

Example: "User prefers semantic search for finding similar skills."
Example: "User frequently checks for survivors in the Swamp Biome."
""",
        )
    ),
    # Topic 2: Urgent Needs Context
    MemoryTopic(
        custom_memory_topic=CustomMemoryTopic(
            label="urgent_needs_context",
            description="""Track the user's focus on urgent needs and resource shortages.

Include:
- Specific resources they are monitoring (food, medicine, ammo)
- Critical situations they are tracking
- Survivors they are particularly concerned about

Example: "User is monitoring the medicine shortage in the Northern Camp."
Example: "User is looking for a doctor for the injured survivors."
""",
        )
    )
]
```

4. 代理程式整合

代理程式碼必須瞭解 Memory Bank，才能儲存及擷取資訊。

👉 在終端機中執行下列指令，在 Cloud Shell 編輯器中開啟檔案：

```
cloudshell edit ~/way-back-home/level_2/backend/agent/agent.py
```

系統會在編輯器中開啟 `~/way-back-home/level_2/backend/agent/agent.py`。

建立代理程式

建立代理時，我們會傳遞 `after_agent_callback`，確保互動後工作階段會儲存至記憶體。`add_session_to_memory` 函式會以非同步方式執行，避免拖慢對話回覆速度。

👉 在 `agent.py` 檔案中，找出 `# TODO: REPLACE_ADD_SESSION_MEMORY` 註解，將整行程式碼替換為下列程式碼：

```
async def add_session_to_memory(
    callback_context: CallbackContext
) -> Optional[types.Content]:
    """Automatically save completed sessions to memory bank in the background"""
    if hasattr(callback_context, "_invocation_context"):
        invocation_context = callback_context._invocation_context
        if invocation_context.memory_service:
            # Use create_task to run this in the background without blocking the response
            asyncio.create_task(
                invocation_context.memory_service.add_session_to_memory(
                    invocation_context.session
                )
            )
        logger.info("Scheduled session save to memory bank in background")
```

背景儲存

👉 在 `agent.py` 檔案中，找出 `# TODO: REPLACE_ADD_MEMORY_BANK_TOOL` 註解，將整行程式碼替換為下列程式碼：

```
if USE_MEMORY_BANK:
    agent_tools.append(PreloadMemoryTool())
```

👉 在 `agent.py` 檔案中，找出 `# TODO: REPLACE_ADD_CALLBACK` 註解，將整行程式碼替換為下列程式碼：

```
after_agent_callback=add_session_to_memory if USE_MEMORY_BANK else None
```

設定 Vertex AI Session Service

👉 🖥️ 在終端機中執行下列指令，以便在 Cloud Shell 編輯器中開啟 `chat.py` 檔案：

```
cloudshell edit ~/way-back-home/level_2/backend/api/routes/chat.py
```

👉 在 `chat.py` 檔案中，找出註解 `# TODO: REPLACE_VERTEXAI_SERVICES`，將整行程式碼替換為下列程式碼：

```
session_service = VertexAiSessionService(
    project=project_id,
    location=location,
    agent_engine_id=agent_engine_id
)
memory_service = VertexAiMemoryBankService(
    project=project_id,
    location=location,
    agent_engine_id=agent_engine_id
)
```

步驟 13 · 約 0 分鐘

13. ☕ [選用] 將代理程式與 Agent Engine 連結

1. 設定及部署

測試記憶體功能前，請先部署含有新記憶體主題的代理程式，並確保環境設定正確無誤。

我們提供便利的指令碼來處理這項程序。

執行部署指令碼

👉 🖥️ 在終端機中執行部署指令碼：

```
cd ~/way-back-home/level_2
./deploy_and_update_env.sh
```

這段指令碼會執行下列動作：

- 執行 `backend/deploy_agent.py`，向 Vertex AI 註冊代理和記憶體主題。
- 擷取新的 **Agent Engine ID**。
- 使用 `AGENT_ENGINE_ID` 自動更新 `.env` 檔案。
- 確認 `USE_MEMORY_BANK=TRUE` 已在 `.env` 檔案中設定。

[!IMPORTANT] 如果您在 `deploy_agent.py` 中變更 `custom_topics`，請務必重新執行這個指令碼，更新 Agent Engine。

驗證 Memory Bank

現在你可以教導代理偏好設定，並確認偏好設定是否會在不同工作階段中保留，藉此驗證記憶庫是否正常運作。

步驟一：開啟應用程式

按照下列指示再次開啟應用程式：如果先前的終端機仍在執行，請按 `Ctrl+C` 結束。

👉 🖥️ 啟動應用程式：

```
cd ~/way-back-home/level_2/
./start_app.sh
```

👉 按一下終端機中的「Local: http://localhost:5173/」。

步驟二：使用文字測試 Memory Bank

在對話介面中，向代理程式說明具體情況：

```
"I'm planning a medical rescue mission in the mountains. I need survivors with first aid and climbing skills."
```

👉 等待約 30 秒，讓記憶體在背景處理。

步驟 3：開始新工作階段

重新整理頁面即可清除目前的對話記錄 (短期記憶)。

根據您先前提供的內容提出問題：

```
"What kind of missions am I interested in?"
```

預期回覆：

「根據您先前的對話，您對以下內容感興趣：

- 醫療救援任務
- 山區/高海拔作業
- 必要技能：急救、攀岩

要我尋找符合這些條件的倖存者嗎？"

步驟 4：使用圖片上傳功能進行測試

上傳圖片並提出以下問題：

```
remember this
```

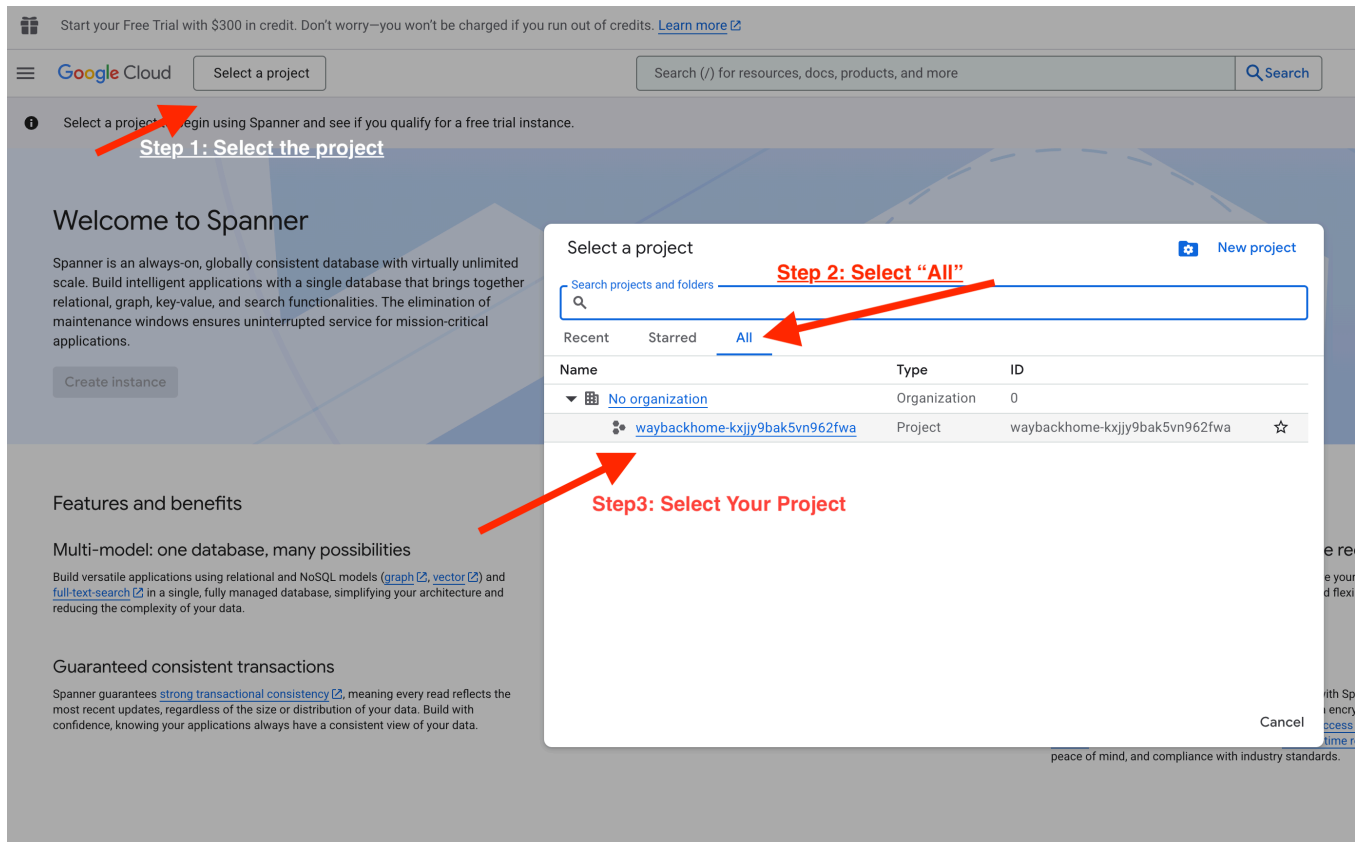
你可以選擇這裡的任何相片或自己的相片，然後上傳至使用者介面：

- [test_photo1](#),
- [test_photo2](#),
- [test_photo3](#)、
- [test_photo4](#)

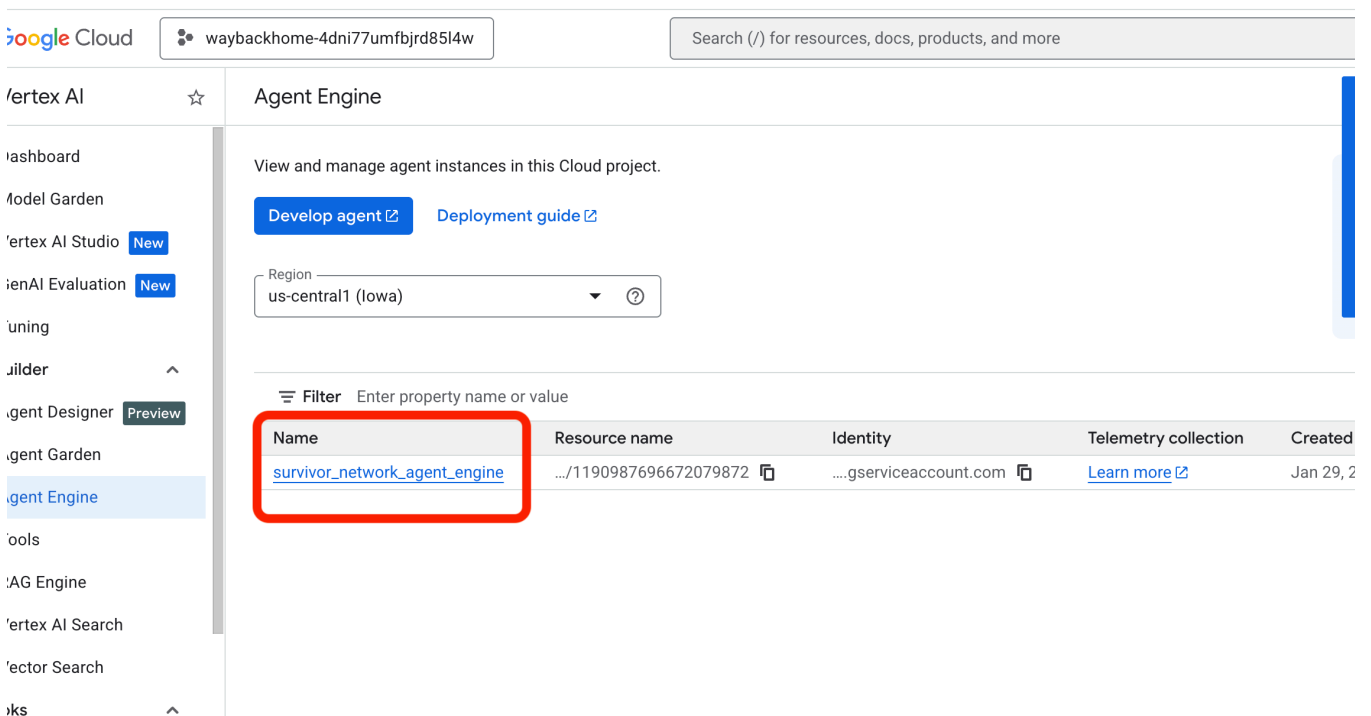
步驟五：在 Vertex AI Agent Engine 中驗證

前往 [Google Cloud 控制台 Agent Engine](#)

1. 請務必從左上方的專案選取器選取專案：



2. 驗證您剛從先前指令 `use_memory_bank.sh` 部署的代理程式引擎：



按一下您剛建立的代理程式引擎。

3. 點選已部署代理中的「**Memories**」分頁，即可查看所有記憶。

Google Cloud

waybackhome-4dni77umfbjrd85i4w

Search (/) for resources, docs, products, and more

Search

✦ 📄 🔔 ⓘ ⋮ 🌐

Vertex AI

← survivor_network_agent_engine

Service Configuration

Copy query URL

Copy identity

Dashboard

Model Garden

Vertex AI Studio New

GenAI Evaluation New

Tuning

Agent Builder ^

Agent Designer Preview

Agent Garden

Agent Engine

Tools

RAG Engine

Vertex AI Search

Vector Search

notebooks ^

Colab Enterprise

Workbench

Dashboard

Runtime Metrics

Traces

Sessions

Playground

Memories

Evaluation

Memory Bank stores long-term memories, which are personalized information, to enable more context-aware agent interactions across multiple sessions. Here you can view, search, and manage the agent's saved memories.

Memories Refresh

Filter Scope: key1:value1 key2:value2 ...

Scope	Resource Name	Fact	Created	Updated	
app_name : survivor-network user_id : test-user	.../2401373252359290880	I have a cat with grey fur and amber ...	Jan 29, 2026, 12:55:24AM	Jan 29, 2026, 12:56:59AM	⋮
app_name : survivor-network user_id : test-user	.../7643563218618548224	I am planning a medical rescue missi...	Jan 29, 2026, 12:47:29AM	Jan 29, 2026, 12:54:39AM	⋮

👉🖥️ 測試完成後，在終端機中按一下「Ctrl + C」結束程序。

🎉 恭喜！您已將記憶庫附加到代理程式！

步驟 14 · 約 0 分鐘

14. ☕ [選用] 部署至 Cloud Run

1. 執行部署指令碼

👉 🖥️ 執行部署指令碼：

```
cd ~/way-back-home/level_2
./deploy_cloud_run.sh
```

部署成功後，您會取得網址，這就是您的部署網址！

```
Waiting for build to complete. Polling interval: 1 second(s).
ID: afcf35bf-3a0f-49cb-81d8-bf67b076342a
CREATE_TIME: 2026-01-29T09:26:04+00:00
DURATION: 5M31S
SOURCE: gs://waybackhome-4dni77umfbjrd85l4w_cloudbuild/source/1769678582.901274-2a58043b40dc44adb5432ec6948a54ac.tgz
IMAGES: -
STATUS: SUCCESS

-----
Cloud Build Submitted Successfully!
Cloud Build Submitted Successfully!
Retrieving Frontend URL...

-----
Deployment Complete!
Frontend URL: https://survivor-frontend-fqq4rdxbdq-uc.a.run.app
```



👉 🖥️ 擷取網址前，請先執行下列指令授予權限：

```
source .env && gcloud run services add-iam-policy-binding survivor-frontend --region $REGION
--member=allUsers --role=roles/run.invoker && gcloud run services add-iam-policy-binding
survivor-backend --region $REGION --member=allUsers --role=roles/run.invoker
```

前往已部署的網址，您會看到應用程式已上線！

2. 瞭解建構管道

`cloudbuild.yaml` 檔案定義了下列循序步驟：

1. 後端建構：從 `backend/Dockerfile` 建構 Docker 映像檔。
2. 後端部署：將後端容器部署至 Cloud Run。
3. 擷取網址：取得新的後端網址。
4. 前端建構：
 - 安裝依附元件。
 - 建構 React 應用程式，並插入 `VITE_API_URL=`。
5. 前端映像檔：從 `frontend/Dockerfile` 建構 Docker 映像檔 (封裝靜態資產)。
6. 前端部署：部署前端容器。

3. 驗證部署作業

建構完成後 (請查看指令碼提供的記錄連結)，您可以驗證：

1. 前往 [Cloud Run 控制台](#)。
2. 找出 `survivor-frontend` 服務。
3. 按一下網址即可開啟應用程式。
4. 執行搜尋查詢，確保前端可以與後端通訊。

(選用) 4. 手動部署

如果您想手動執行指令或進一步瞭解程序，請參閱下文，瞭解如何直接使用 `cloudbuild.yaml`。

撰寫 `cloudbuild.yaml`

`cloudbuild.yaml` 檔案會告知 Google Cloud Build 要執行的步驟。

- **steps**：循序動作清單。每個步驟都會在容器中執行 (例如 `docker`、`gcloud`、`node`、`bash`)。
- **替代**：可在建構時間傳遞的變數 (例如 `$_REGION`)。
- **工作區**：共用目錄，步驟可以在其中共用檔案 (就像我們共用 `backend_url.txt` 一樣)。

執行 Deployment

如要手動部署，請使用 `gcloud builds submit` 指令。您「必須」傳遞必要的替代變數。

```
# Load your env vars first or replace these values manually
export PROJECT_ID=your-project-id
export REGION=us-central1

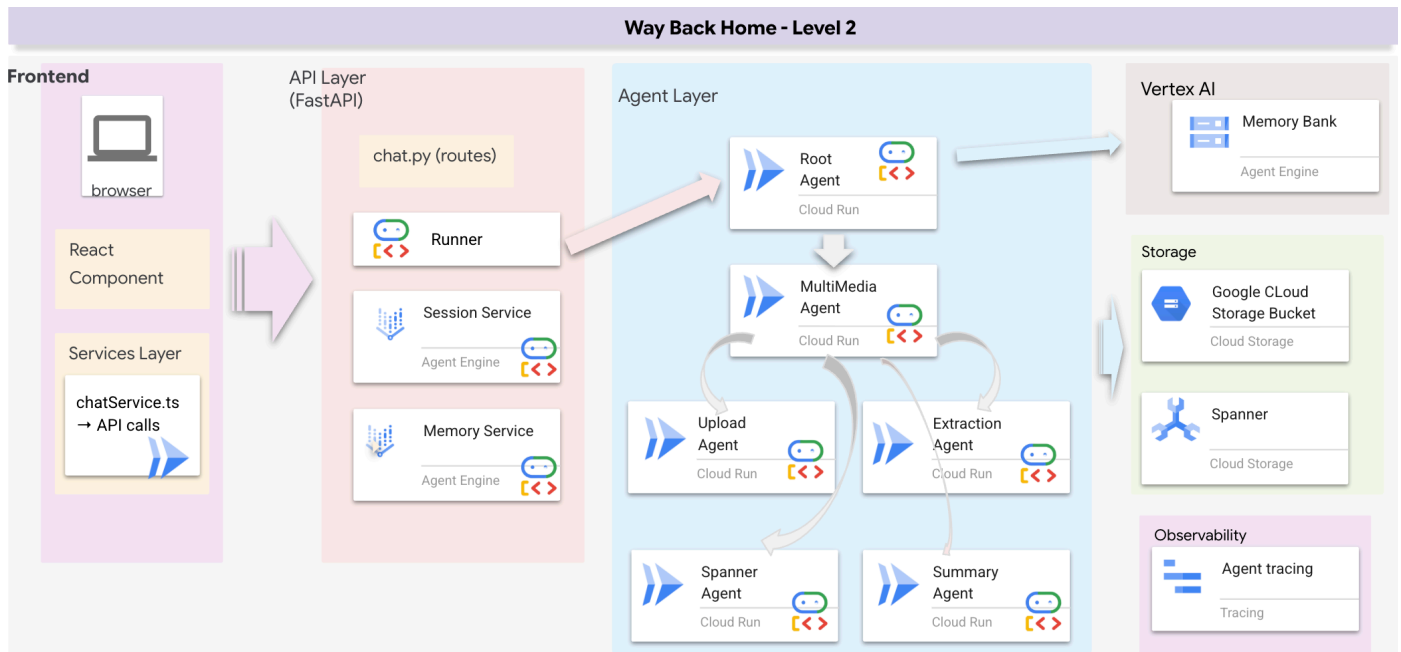
gcloud builds submit --config cloudbuild.yaml \
  --project "$PROJECT_ID" \
  --substitutions _REGION="us-central1",_GOOGLE_API_KEY="",_AGENT_ENGINE_ID="your-agent-id",_USE_MEMORY_BANK="TRUE",_GOOGLE_GENAI_USE_VERTEXAI="TRUE"
```

15. 結論

1. 建構項目

- ✓ 圖形資料庫：Spanner，包含節點 (倖存者、技能) 和邊緣 (關係)
- ✓ AI 搜尋：關鍵字、語意和混合搜尋 (含嵌入)
- ✓ 多模態管道：使用 Gemini 從圖片/影片中擷取實體
- ✓ 多代理程式系統：使用 ADK 協調工作流程
- ✓ Memory Bank：使用 Vertex AI 進行長期個人化
- ✓ 正式版部署：Cloud Run + Agent Engine

2. 架構摘要



3. 學習要點

1. 圖表 RAG：結合圖表資料庫結構與語意嵌入，提供智慧型搜尋功能
2. 多代理模式：適用於複雜多步驟工作流程的循序管道
3. 多模態 AI：從非結構化媒體 (圖片/影片) 擷取結構化資料
4. 有狀態的代理：Memory Bank 可跨工作階段提供個人化服務

4. 研討會內容

- Level0：驗證身分
- Level1：精確位置
- Level2 This One：使用 Graph RAG、ADK 和 Memory Bank 建構多模態 AI 代理程式
- Level3：建構 ADK 雙向串流代理
- Level4：即時雙向多代理系統
- Level5：使用 Google ADK、A2A 和 Kafka 的事件導向架構

5. 資源

- [ADK 說明文件](#)
- [Spanner Graph](#)
- [Memory Bank 指南](#)
- [GitHub 存放區](#)

恭喜！ 您已建構正式環境等級的 AI 代理程式系統，其中包含圖形資料庫、RAG 搜尋、多模態處理和長期記憶體。這些模式適用於任何需要智慧資料協調的領域，包括物流、醫療保健和客戶服務。
